



Inherently interpretable machine learning for credit scoring: Optimal classification tree with hyperplane splits

Jiancheng Tu^{a,b}, Zhibin Wu^{a,*}

^a Business School, Sichuan University, Chengdu 610065, China

^b Department of Computing, Hong Kong Polytechnic University, Hong Kong

ARTICLE INFO

Keywords:

Decision support systems
Interpretable machine learning
Optimal classification tree
Credit scoring
Cost-sensitive learning

ABSTRACT

An accurate and interpretable credit scoring model plays a crucial role in helping financial institutions reduce losses by promptly detecting, containing, and preventing defaulters. However, existing models often face a trade-off between interpretability and predictive accuracy. Traditional models like Logistic Regression (LR) offer high interpretability but may have limited predictive performance, while more complex models may improve accuracy at the expense of interpretability. In this paper, we tackle the credit scoring problem with imbalanced data by proposing two new classification models based on the optimal classification tree with hyperplane splits (OCT-H). OCT-H provides transparency and easy interpretation with ‘if-then’ decision tree rules. The first model, the cost-sensitive optimal classification tree with hyperplane splits (CSOCT-H). The second model, the optimal classification tree with hyperplane splits based on maximizing F1-Score (OCT-H-F1), aims to directly maximize the F1-score. To enhance model scalability, we introduce a data sample reduction method using data binning and feature selection. We then propose two solution methods: a heuristic approach and a method utilizing warm-start techniques to accelerate the solving process. We evaluated the proposed models on four public datasets. The results show that OCT-H significantly outperforms traditional interpretable models, such as Decision Trees (DT) and Logistic Regression (LR), in both predictive performance and interpretability. On certain datasets, OCT-H performs as well as or better than advanced ensemble tree models, effectively narrowing the gap between interpretable models and black-box models.

1. Introduction

Credit scoring models are vital tools used to differentiate between creditworthy and risky loan applicants (Siddiqi, 2012; Thomas, Crook, & Edelman, 2002). Their significance has grown in recent years, driven by the expansion of the lending industry and the need for lenders to swiftly and accurately evaluate the creditworthiness of potential borrowers. The Federal Reserve data reveals that consumer credit in the United States surpassed \$4,785.7 billion in 2022 and continues to grow at an annual rate exceeding 4% (<https://www.federalreserve.gov/releases/g19/current/default.htm>). In 2016, according to the European Central Bank, fraudulent transactions involving cards issued within the Single Euro Payments Area (SEPA) and acquired worldwide reached a staggering value of €1.8 billion. Consequently, developing powerful credit scoring models is crucial for financial institutions to proactively prevent, manage, and mitigate losses stemming from defaulters.

Over the years, numerous credit scoring models based on statistical and machine learning methods have been developed (Dumitrescu, Hué, Hurlin, & Tokpavi, 2022; Gunnarsson, vanden Broucke, Baesens,

Óskarsdóttir, & Lemahieu, 2021; Lessmann, Baesens, Seow, & Thomas, 2015; Liu, Baals, Osterrieder, & Hadji-Misheva, 2024; Liu, Fan, & Xia, 2023; Luo, Yan, & Tian, 2020; Medina-Olivares, Calabrese, Crook, & Lindgren, 2023). These models fall into two categories: interpretable models (such as decision trees and logistic regression) and black-box models (such as random forests, boosted decision trees, and neural networks).

Interpretability is a key requirement for credit scoring models (Molnar, 2020). It is essential not only for providing explanations to customers whose credit is denied but also for fostering trust and transparency. Complex models, like ensemble methods and deep learning, are often viewed with skepticism and distrust by both users and regulators (Baesens & Smedts, 2023). The Financial Stability Board (FSB) has noted that the use of complex algorithms can result in a lack of transparency for consumers, raising concerns about the black-box nature of machine learning algorithms (Financial Stability Board, 2017). Consequently, despite the superior accuracy of modern machine learning

* Corresponding author.

E-mail addresses: jiancheng.tu@connect.polyu.hk (J. Tu), zhibinwuscu@qq.com, zhibinwu@scu.edu.cn (Z. Wu).

techniques, logistic regression remains the industry standard in credit scoring (Bücker, Szepannek, Gosiewska, & Biecek, 2022).

Black-box machine learning methods, including random forests (Breiman, 2001), boosted decision trees (Chang, Chang, & Wu, 2018), and neural networks (Kellner, Nagl, & Rösch, 2022), have shown significant accuracy improvements over interpretable models like decision trees and logistic regression (Lessmann et al., 2015; Liu et al., 2023). However, the internal workings of black-box models are opaque, making it difficult for evaluators to understand how scores or ratings are generated, which undermines their credibility and reliability.

Therefore, choosing between a black-box model with high accuracy and an interpretable model with lower accuracy often presents a dilemma (Wang & Lin, 2021).

Credit scoring models often face the challenge of class imbalance (CI) (He & Garcia, 2009), primarily due to the scarcity of default records (Jiang, Lu, Wang, & Ding, 2023). Most traditional binary classification models aim to maximize classification accuracy. However, accuracy only considers the number of correctly classified cases while assuming equal costs for type I and type II errors. In many real-world scenarios, these costs are not equal, making accuracy an inadequate measure of model performance (Chaabane, Guermazi, & Hammami, 2020).

To address these issues, we propose two classification models based on the Optimal Classification Tree with Hyperplane Splits (OCT-H). These models are interpretable due to their ‘if-then’ decision rules and demonstrate high predictive performance on imbalanced credit scoring datasets compared to classical decision trees. The first model, the Cost-Sensitive Optimal Classification Tree with Hyperplane Splits (CSOCT-H), incorporates predefined class weights to address classification errors. The second model, the Optimal Classification Tree with Hyperplane Splits based on maximizing F1-Score (OCT-H-F1), directly maximizes the F1-score.

The rest of this paper is organized as follows: Section 2 reviews optimal classification trees and machine learning methods in credit scoring. Section 3 introduces a new OCT-H model and its two variants: OCT-H-F1 and CSOCT-H. Section 4 presents efficient methods to enhance the scalability of the proposed models on large datasets. Section 5 presents the results of empirical evaluations for the proposed approaches. Conclusions are given in Section 6.

2. Related work

2.1. Optimal classification tree

Classification trees are a commonly used technique for solving classification problems. They recursively partition the feature space based on the training data and assign labels to each resulting partition. This tree structure can then be used to classify future data points based on these partitions and labels. One of the key advantages of using decision trees is their interpretability, making them preferable in high stakes decisions such as healthcare and risk management, where interpretability is valued over higher accuracy but less interpretable methods (Rudin, 2019). Therefore, there has been increasing attention towards the use of decision trees for decision-making problems due to their interpretability in recent literature and practice (Aouad, Elmachetoub, Ferreira, & McNellis, 2023; Ciocan & Mišić, 2022).

The classification and regression trees (CART) method, proposed by Breiman, Friedman, Olshen, and Stone (1984), is considered the leading approach for decision tree methods in classification. CART adopts a top-down strategy for partition determination. However, a major limitation of this approach, is its inherently greedy nature. In these methods, each split in the decision tree is determined independently without considering the potential influence of future splits. Consequently, the resulting trees may not effectively capture the underlying characteristics of the dataset, resulting in suboptimal performance

when classifying future data points. Another disadvantage of heuristic-based approaches is their difficulty in training multivariate or oblique decision trees, where splits at a node involve multiple variables or hyperplanes. Additionally, these heuristic methods do not easily accommodate desirable global structural properties, such as model sparsity and cost-sensitivity, due to their greedy nature.

The use of top-down induction and pruning in CART algorithm was not based on a belief that this approach was inherently superior. Instead, it was driven by practical considerations at that time, as finding an optimal tree is a non-deterministic polynomial hard (NP-hard) problem (Laurent & Rivest, 1976). With the continuous improvements in software and hardware, certain solvers such as Gurobi (Gurobi Optimization, 2024) and CPLEX (IBM, 2023) now have the capability to efficiently solve the mixed-integer programming (MIP) problems on a large scale. Therefore, there has been a growing number of studies utilizing different mathematical optimization methods to construct the optimal decision trees (Aghaei, Gómez, & Vayanos, 2024; Bertsimas & Dunn, 2017; Blanquero, Carrizosa, Molero-Río, & Morales, 2020; Carrizosa, Molero-Río, & Romero Morales, 2021).

The most relevant work in relation to our study is the optimal classification tree proposed by Bertsimas and Dunn (2017) based on MIP. In their work, they developed two models for optimal sparse classification trees: one for axes-aligned splits and another for multivariate splits (also called optimal classification trees with hyperplanes). Bertsimas and Dunn (2017) demonstrated that the prediction accuracy of the optimal classification tree with hyperplanes (OCT-H) surpasses that of CART. Notably, OCT-H achieves comparable performance to random forests when the number of classes is less than 3 and the number of features is less than 25, which indicates that OCT-H is very appropriate for credit scoring problems. However, due to its NP-hard nature, this OCT-H model is not scalable and struggles with large datasets commonly encountered in credit scoring, often exceeding 10,000 samples.

2.2. Machine learning models in credit scoring

2.2.1. Interpretable machine learning in credit scoring

There is little consensus on defining and measuring interpretability in machine learning (Doshi-Velez & Kim, 2017). Many scholars use ‘interpretable’ and ‘explainable’ interchangeably (Doshi-Velez & Kim, 2017; Molnar, 2020). Consistent with Rudin (2019), this paper defines interpretable machine learning as models inherently interpretable, such as decision trees and logistic regression. In contrast, methods providing post-hoc explanations for black-box models are termed ‘explainable machine learning’.

Interpretable models emphasize low complexity and simple logic. Model complexity relates to the cognitive load required for understanding (Doshi-Velez & Kim, 2017). Simple logic facilitates human reasoning and comprehension of the predictive process. Interpretable models used in credit scoring include multiple discriminant analysis (Altman, 1968), quantile regression (Miguéis, Benoit, & Van den Poel, 2013), logistic regression (Andreeva & Altman, 2021), generalized additive models (Lohmann & Ohliger, 2019), survival analysis (Bellotti & Crook, 2009; Roza, Crook, & Andreeva, 2023), Bayesian learning (Wang, Crook, & Andreeva, 2020), and decision trees (Teles, Rodrigues, Saleem, Kozlov, & Rabêlo, 2020). However, these models often lag behind black-box models in predictive performance.

Recent efforts aim to design models that are both interpretable and accurate for financial risk management (Baesens, Höppner, & Verdonck, 2021; Chen, Lin, Rudin, Shaposhnik, Wang, & Wang, 2022; Dumitrescu et al., 2022). Baesens et al. (2021) showed that with effective data engineering, simple models like logistic regression and classification trees can excel in fraud detection. Chen et al. (2022) developed a globally transparent two-layer additive risk model for credit scoring, outperforming black-box models like random forests and neural networks. Dumitrescu et al. (2022) enhanced logistic regression by integrating decision tree information, achieving performance comparable to random forests on some datasets. These models demonstrate the potential for designing interpretable and accurate models, though further validation is needed for scalability and generalizability.

2.2.2. Explainable machine learning in credit scoring

Explainable machine learning aims to provide explanations for black-box machine learning models, such as SHapley Additive exPlanation (SHAP) (Lundberg & Lee, 2017), Local Interpretable Model-agnostic Explanations (LIME) (Ribeiro, Singh, & Guestrin, 2016), Local Rule-Based Explanations (LORE) (Guidotti et al., 2018) and Anchors (Ribeiro, Singh, & Guestrin, 2018). These models are widely applied in the credit scoring (Chen, Calabrese, & Martin-Barragan, 2024; Demajo, Vella, & Dingli, 2020; Moscato, Picariello, & Sperli, 2021).

However, post-hoc explainable models face concerns over inconsistencies (Rudin, 2019), susceptibility to manipulation (Slack, Hilgard, Jia, Singh, & Lakkaraju, 2020), and instability (Ross, Hughes, & Doshi-Velez, 2017). In credit scoring, discrepancies may arise between explanations for loan denials and actual model computations, as explanations are created post-hoc (Chen et al., 2022; Rudin, 2019). Additionally, interpretations from LIME and SHAP become less stable with increasing class imbalance (Chen et al., 2024).

2.2.3. Cost-sensitive machine learning in credit scoring

Cost-sensitive learning is a type of learning where misclassification cost is used to adjust the learning process by synthetically balancing the frequencies of the two classes, recognizing that misclassification costs from type I and type II errors differ (Yotsawat, Wattuya, & Srivihok, 2021). In the context of credit scoring, the term “misclassification costs” can be interpreted in two distinct ways.

First, misclassification costs can refer to class weights used in machine learning algorithms to address class imbalance. In this context, cost-sensitive machine learning involves assigning different weights to different classes (e.g., assigning a higher weight to the minority class of bad applicants) during the training process. By treating class weights as a hyperparameter and adjusting them, models aim to improve performance metrics such as the Area Under the ROC Curve (AUC) and the F1-score (Iranmehr, Masnadi-Shirazi, & Vasconcelos, 2019; Krawczyk, Woźniak, & Schaefer, 2014; Liu, Fan, Xia, & Xia, 2022; Lomax & Vadera, 2013; Yotsawat et al., 2021). However, determining appropriate weights is challenging, as incorrect settings can lead to misclassifications or bias towards common classes.

Second, misclassification costs can denote the actual financial costs associated with different types of errors—specifically, the economic losses resulting from misclassifying good applicants as bad (false positives) or bad applicants as good (false negatives) (Baesens et al., 2003; Kim, Choi, Kim, & Suh, 2012; Lohmann & Ohliger, 2019). In this approach, models incorporate estimates of the financial costs of these two types of errors directly into the training process, aiming to minimize the expected financial loss. Kim et al. (2012) compared traditional and cost-sensitive classifiers, showing significant cost reductions with cost-sensitive classifiers. Some scholars have found that the financial costs associated with misclassification vary across different samples (Bahnsen, Aouada, & Ottersten, 2014; Höppner, Baesens, Verbeke, & Verdonck, 2022). Consequently, they have proposed credit scoring models that utilize instance-dependent cost-sensitive strategies. These models require estimating the financial costs for each sample to train models that minimize overall financial costs.

While there is no consensus on determining the final financial costs for each type of error, it is widely recognized that extending credit to risky applicants is costlier than denying credit to reliable ones (Baesens et al., 2003). The cost of a false negative — classifying a bad applicant as good — equals the economic loss incurred from default (Petrides, Moldovan, Coenen, Guns, & Verbeke, 2022; Verbraken, Bravo, Weber, & Baesens, 2014). Kim et al. (2012) suggested collaborating with domain experts to establish appropriate financial costs.

2.3. Contributions

As previously mentioned, choosing between a black-box model with high accuracy and an interpretable model with lower accuracy often

Table 1

Confusion matrix.

		Prediction	
		Bad	Good
Label	Bad	TP	FN
	Good	FP	TN

presents a dilemma (Wang & Lin, 2021). Our research aims to develop models that are both interpretable and accurate for credit scoring problems. The main contributions of this paper are:

1. We propose a new MIP formulation of OCT-H, which offers complete transparency and ease of interpretation through its ‘if-then’ decision tree rules. Our model is more flexible than that of Bertsimas and Dunn (2017), allowing for various objectives and constraints. To address data imbalance in credit scoring, we introduce two variants: Cost-Sensitive OCT-H (CSOCT-H), which assigns greater weight to the minority class, and OCT-H based on maximizing F1-Score (OCT-H-F1), which aims to directly maximize the F1-score.
2. Finding the optimal classification tree is an NP-hard problem (Laurent & Rivest, 1976). Additionally, OCT-H faces scalability challenges, particularly with datasets larger than 10,000 samples. In Section 4, we introduce a data sample reduction method based on data binning and feature selection. We then propose two solution methods to accelerate the solving process. Experiments demonstrate that our methods can handle large datasets with more than 770,000 instances.
3. Experimental results indicate that our model outperforms tree-based ensemble models, including Random Forest (RF), LightGBM, and XGBoost, in terms of F1-score. Additionally, it surpasses RF in AUC, while demonstrating comparable performance with LightGBM and XGBoost in AUC on certain datasets.

3. Model and problem formulation

In this section, we first introduce the OCT-H. We then present its two main variants, namely, Cost-Sensitive OCT-H (CSOCT-H) and OCT-H based on maximizing F1-score (OCT-H-F1), designed to address imbalanced credit scoring datasets.

Let $I = \{(\mathbf{x}_i, y_i)\}_{i=1}^{|I|}$ denote the training credit set. Here, $\mathbf{x}_i = (x_{i1}, x_{i2}, \dots, x_{i|F|}) \in \mathbb{R}^{|F|}$ is an $|F|$ -dimensional vector containing the applicant’s characteristics, where each feature x_{if} is normalized by dividing by the maximum value of the f th feature. The binary variable $y_i \in \{0, 1\}$ distinguishes between good ($y_i = 0$) and bad ($y_i = 1$) applicants.

Table 1 presents the confusion matrix for credit scoring, which includes the following elements: TP (True Positives), the count of accurately predicted bad applicants; FN (False Negatives), the count of incorrectly predicted bad applicants; FP (False Positives), the count of good applicants wrongly classified; and TN (True Negatives), the count of accurately predicted good applicants.

To assess the performance of binary classification models, the following four evaluation criteria are widely used: accuracy (Acc), precision, recall, and F1-score.

$$\text{Acc} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FN} + \text{FP} + \text{TN}} \quad (1)$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (2)$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (3)$$

$$\text{F1-score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4)$$

Table 2
Summary of notation.

Notation	Definition
Set	
I	Set of training data
F	Set of features
K	Set of labels
B	Set of branch nodes in the decision tree
L	Set of leaf nodes in the decision tree
$A(t)$	Set of ancestors of node t in the decision tree
$A_R(t)$	Set of right-branch ancestors of node t
$A_L(t)$	Set of left-branch ancestors of node t
$L_P(t)$	Set of potential parent nodes of leaf node t
$L_R(t)$	Set of potential right leaf nodes of branch node t
$L_L(t)$	Set of potential left leaf nodes of branch node t
Index	
i	Instance in training data, $i \in I$
f	Feature in training data, $f \in F$
k	Label in training data, $k \in K$
t	Node of tree, $t \in B \cup L$
$p(t)$	Parent of node t
Parameter	
D	the depth of the tree
α	the complexity parameter
$S_{\min}$	minimum number of instances at each leaf node
N	the maximum number of features used in a branch node

We introduce the binary variable z_{it} to represent misclassification errors. Specifically, $z_{it} = 1$ if instance i is assigned to leaf node t and the prediction of instance i 's label is correct; otherwise, $z_{it} = 0$. Let n^+ denote the number of positive samples and n^- denote the number of negative samples in the training set. TP, FN, FP, and TN can be expressed in terms of the decision variable z_{it} as follows:

$$TP = \sum_{i \in L} \sum_{i \in I: y_i=1} z_{it} \quad (5)$$

$$FN = n^+ - \sum_{i \in L} \sum_{i \in I: y_i=1} z_{it} \quad (6)$$

$$FP = n^- - \sum_{i \in L} \sum_{i \in I: y_i=0} z_{it} \quad (7)$$

$$TN = \sum_{i \in L} \sum_{i \in I: y_i=0} z_{it} \quad (8)$$

Therefore, we can apply the binary variable to express any evaluation metrics computed by TP, FN, FP, and TN, such as accuracy, recall, precision, F1-score, and cost-sensitive objectives. In this paper, we present three different versions of the OCT-H, namely, optimal classification tree with hyperplane splits based on maximizing accuracy (Section 3.1), CSOCT-H (Section 3.2) and OCT-H-F1 (Section 3.3).

3.1. Optimal classification tree with hyperplane splits based on maximizing accuracy

According to Eq. (1), accuracy can be expressed by z_{it} as follows:

$$Acc = \left(\sum_{i \in L} \sum_{i \in I: y_i=1} z_{it} + \sum_{i \in L} \sum_{i \in I: y_i=0} z_{it} \right) / |I| \quad (9)$$

We introduce the binary variable s_{tf} to measure the complexity of the model, where $s_{tf} = 1$ if feature f is selected to split at branch node t and $s_{tf} = 0$ otherwise. The objective is to maximize accuracy while minimizing the complexity of the tree, which is measured by the number of features used. This can be formulated as follows:

$$J = \left(\sum_{i \in L} \sum_{i \in I: y_i=1} z_{it} + \sum_{i \in L} \sum_{i \in I: y_i=0} z_{it} \right) / |I| - \alpha \sum_{f \in F} \sum_{t \in B} s_{tf} \quad (10)$$

where α is the parameter used to control the complexity of the tree, usually set to 0.01. The implication of Eq. (10) is that if adding a feature does not reduce the average misclassification error of the model by

Table 3
Summary of decision variables.

Notation	Definition
a_{tf}	The coefficient of feature f at node t , $a_{tf} \in [-1, 1]$
$\hat{a}_{tf}$	Auxiliary variables to track the value of $ a_{tf} $, $\hat{a}_{tf} \in [0, 1]$
b_t	The threshold at branch node t , $b_t \in [-1, 1]$
c_{tk}	If label k is assigned to leaf node t ($=1$) or not ($=0$)
d_t	If a split rule is applied to branch node t ($=1$) or not ($=0$)
l_t	If leaf node t has at least $S_{\min}$ instances t ($=1$) or not ($=0$)
z_{it}	If instance i is assigned to leaf node t ($=1$) or not ($=0$)
s_{tf}	If feature f is selected to split at branch node t ($=1$) or not ($=0$)
e_{ik}	If instance i is predicted as label k ($=1$) or not ($=0$)

more than α , then this feature will be discarded. All the notations and decision variables used in this paper are presented in Tables 2 and 3, respectively.

The MIP formulation of the optimal classification tree with hyperplane splits for maximizing accuracy (OCT-H-Acc) is provided as follows.

(OCT-H-Acc):

$$\begin{aligned} \max \quad & \left(\sum_{i \in L} \sum_{i \in I: y_i=1} z_{it} + \sum_{i \in L} \sum_{i \in I: y_i=0} z_{it} \right) / \\ & |I| - \alpha \sum_{f \in F} \sum_{t \in B} s_{tf} \\ & \sum_{f \in F} \hat{a}_{tf} \leq d_t, \quad \forall t \in B \quad (11a) \\ & a_{tf} \leq \hat{a}_{tf}, \quad \forall f \in F, \forall t \in B \quad (11b) \\ & -a_{tf} \leq \hat{a}_{tf}, \quad \forall f \in F, \forall t \in B \quad (11c) \\ & \hat{a}_{tf} \leq s_{tf}, \quad \forall f \in F, \forall t \in B \quad (11d) \\ & s_{tf} \leq d_t, \quad \forall f \in F, \quad \forall t \in B \quad (11e) \\ & \sum_{f \in F} s_{tf} \geq d_t, \quad \forall t \in B \quad (11f) \\ & -d_t \leq b_t \leq d_t, \quad \forall t \in B \quad (11g) \\ & d_t \leq d_{p(t)}, \quad \forall t \in B \setminus \{1\} \quad (11h) \\ & l_t \leq \sum_{j \in L_{P(t)}} d_j, \quad \forall t \in L \quad (11i) \\ & D l_t \geq \sum_{j \in L_{P(t)}} d_j, \quad \forall t \in L \quad (11j) \\ & \sum_{k \in K} c_{tk} = l_t, \quad \forall t \in L \quad (11k) \\ & \sum_{f \in F} s_{tf} \leq N, \quad \forall t \in B \quad (11l) \\ & z_{it} \leq c_{tk}, \quad k = y_i, \forall i \in I \quad (11m) \\ & z_{it} \leq l_t, \forall i \in I, \quad \forall t \in L \quad (11n) \\ & \sum_{i \in I} z_{it} \geq S_{\min} l_t, \quad \forall t \in L \quad (11o) \end{aligned} \quad (11)$$

$$\begin{aligned} & \sum_{f \in F} x_{if} a_{jf} \geq b_j - 2 \left(1 - \sum_{i \in L_{R(j)}} z_{it} \right), \\ & \forall i \in I, \forall j \in B, \quad (11p) \end{aligned}$$

$$\begin{aligned} & \sum_{f \in F} x_{if} a_{jf} + \mu d_j \leq b_j + (2 + \mu) \left(1 - \sum_{i \in L_L(j)} z_{it} \right), \\ & \forall i \in I, \forall j \in B \quad (11q) \end{aligned}$$

$$a_{tf}, b_t \in [-1, 1], \quad \forall t \in B, \forall f \in F \quad (11r)$$

$$c_{tk}, z_{it}, l_t, d_t \in \{0, 1\}, \quad \forall i \in I, \quad \forall t \in L, \forall k \in K \quad (11s)$$

Let us first introduce the constraints on the tree structure. Constraints (11a)–(11c) ensure that no split is applied at branch node t if the node t does not exist. The auxiliary variables $\hat{a}_{tf}$ are used to track the value of $|a_{tf}|$. If $\hat{a}_{tf} = 0$, constraints (11b)–(11c) ensure that $a_{tf} = 0$. By combining constraint (11d) with the complexity

function $\alpha \sum_{f \in F} \sum_{i \in B} s_{if}$ in the objective function, s_{if} will be equal to zero if and only if the feature f is not used in branch node t . Constraints ((11)e) and ((11)f) ensure that at least one feature will be used to split if the branch node exists ($d_t = 1$). The relationship between b_t and d_t is presented in ((11)g). If the branch node t does not exist ($d_t = 0$), then b_t must be equal to zero; otherwise, b_t belongs to $[-1, 1]$. Constraints ((11)h)–((11)i) express the relationship between l_t and d_t . The prerequisite for the existence of branch node t is that its parent node also exists, as expressed in constraint ((11)h). $L_P(t)$ is the set of potential parent branch nodes for leaf node t . For example, in the tree shown in Fig. 1, $L_P(4) = \{1, 2\}$ and $L_P(5) = \{2\}$. Constraint ((11)i) enforces that there is at least one potential parent branch node when the leaf node t exists; constraint ((11)j) states that the leaf node t must exist when its potential parent branch nodes set is not empty. Constraint ((11)k) ensures that one label will be assigned to a leaf node. To control the complexity of the model, we introduce constraint ((11)l) to ensure that the number of features used in a branch node does not exceed the pre-set value N .

Based on the above constraints for the tree structure, we now introduce constraints on how to partition the dataset. Constraint ((11)m) ensures that $z_{it} = 1$ when the label of leaf node t matches the label of instance i . Constraints ((11)n) and ((11)o) express the relationship between z_{it} and l_t . If $l_t = 0$, then no instance will be assigned to leaf node t . If $l_t = 1$, then the number of instances assigned to leaf node t must be greater than or equal to $S_{\min}$. Constraint ((11)p) ensures that instance i is assigned to the right child leaf t if $\sum_{f \in F} x_{if} a_{sf} \geq b_s$ for all $s \in A_L(t)$. Similarly, constraint ((11)q) ensures that instance i is assigned to the left child leaf t if $\sum_{f \in F} x_{if} a_{sf} < b_s$ for all $s \in A_R(t)$, where $\mu = 0.005$. $A_L(t)$ and $A_R(t)$ are the sets of left-branch and right-branch ancestors of node t , respectively. For example, in the tree shown in Fig. 1, $A_L(4) = \{1, 2\}$, $A_R(4) = \{\}$, $A_L(5) = \{1\}$, and $A_R(5) = \{2\}$.

There are three main differences between the OCT-H-Acc and the model proposed by Bertsimas and Dunn (2017):

1. **Constraints:** OCT-H-Acc adds two constraints ((11)i) and ((11)j) to express the relationship between d_t and l_t . Additionally, we have made some modifications to the constraints in OCT-H. Consider the following situation: if instance i is assigned to leaf node t while node s (one of the left-branch ancestors of node t) does not exist, then the constraint $\sum_{f \in F} x_{if} a_{sf} + \mu \leq b_s + (2 + \mu)(1 - z_{it})$ proposed by Bertsimas and Dunn (2017) is equivalent to $\mu \leq 0$. However, μ is a relatively small positive real number, set to 0.005. To address this issue, we suggest using $\sum_{f \in F} x_{if} a_{sf} + \mu d_s \leq b_s + (2 + \mu)(1 - \sum_{i \in L_L(j)} z_{it})$.
2. **Generalizations:** OCT-H-Acc is much more flexible, allowing for the easy addition of commonly used metrics (recall, precision, and F1-score) to evaluate the predictive performance of imbalanced datasets (see Sections 3.2 and 3.3).
3. **Scalability:** Because $\sum_{i \in L_R(j)} z_{it}$ and $\sum_{i \in L_L(j)} z_{it}$ are greater than or equal to z_{it} , constraints ((11)p) and ((11)q) are stronger than constraints (26) and (27) in the model proposed by Bertsimas and Dunn (2017). Hence, OCT-H-Acc is stronger and requires less time to solve the same problem compared to the model proposed by Bertsimas and Dunn (2017).

3.2. Cost-sensitive optimal classification tree with hyperplane splits

Cost-sensitive machine learning is a method that assigns weights to minority class members to balance class frequencies, effectively addressing imbalanced datasets by enhancing predictive performance on underrepresented classes. In credit scoring, bad samples typically constitute a small proportion of the dataset and misclassifying bad samples can lead to significant risks and losses (Baesens et al., 2003; Lohmann & Ohliger, 2019), making cost-sensitive machine learning a more suitable approach than the traditional machine learning models.

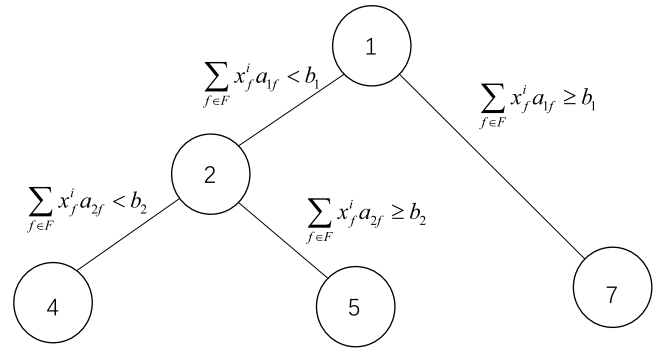


Fig. 1. An example of a decision tree.

According to Eqs. (6) and (7), the objective of the cost-sensitive OCT-H (CSOCT-H) can be written as follows:

$$J = \left[C_{FP} \left(n^- - \sum_{i \in L} \sum_{i \in I: y_i=0} z_{it} \right) + C_{FN} \left(n^+ - \sum_{i \in L} \sum_{i \in I: y_i=1} z_{it} \right) \right] / |I| + \alpha \sum_{f \in F} \sum_{i \in B} s_{if} \quad (12)$$

In credit scoring, the number of negative samples (good applicants) is greater than that of positive samples (bad applicants). Therefore, we set C_{FP} to 1 and C_{FN} to a number greater than 1, with its default value being n^-/n^+ . During the training process, different values of C_{FN} can be tested to synthetically adjust the frequencies of the two classes and enhance the model's performance on evaluation metrics such as the F1-score. Thus, the parameter C_{FN} functions similarly to the `class_weight` parameter in various machine learning models in scikit-learn (Pedregosa et al., 2011), helping to address the class imbalance issue.

The MIP formulation of CSOCT-H is given as follows.

(CSOCT-H):

$$\min \left[C_{FP} \left(n^- - \sum_{i \in L} \sum_{i \in I: y_i=0} z_{it} \right) + C_{FN} \left(n^+ - \sum_{i \in L} \sum_{i \in I: y_i=1} z_{it} \right) \right] / |I| + \alpha \sum_{f \in F} \sum_{i \in B} s_{if} \quad (13)$$

Constraints ((11)a) – ((11)s)

Remark 1. In cost-sensitive machine learning, misclassification cost is used to adjust the learning process by synthetically balancing the frequencies of the two classes (Yotsawat et al., 2021). In the context of credit scoring, misclassification cost refers to the financial cost, which is typically used to evaluate and/or train credit scoring models (Baesens et al., 2003; Höppner et al., 2022; Lohmann & Ohliger, 2019). In this paper, CSOCT-H is applied to enhance predictive performance, as measured by metrics such as AUC and F1-score, by adjusting class weights. However, empirical research indicates that single-item validity measures like AUC and F1-score may yield distorted results by overlooking the total costs associated with incorrect predictions. In contrast, using a measure that incorporates the financial costs associated with misclassification allows for a more accurate assessment of the economic benefits provided by different models (Lohmann & Ohliger, 2019). CSOCT-H addresses this issue by providing flexibility in setting the cost parameters C_{FP} and C_{FN} . In scenarios where C_{FP} and C_{FN} are known and represent specific financial costs, CSOCT-H can directly minimize these costs. This is particularly beneficial when precise cost information is available, as it enables a more financially informed decision-making process.

Remark 2. If instance-dependent costs are known, the model can be extended to an instance-dependent cost-sensitive learning framework, termed the instance-dependent cost-sensitive OCT-H (IDCSOCT-H), as shown in model (14). This approach enables the model to account for varying costs or benefits associated with individual instances, further enhancing its applicability in real-world scenarios.

(IDCSOCT-H):

$$\begin{aligned} \min \quad & \sum_{i \in I} C_i \left(1 - \sum_{t \in L} z_{it} \right) + \alpha \sum_{f \in F} \sum_{t \in B} s_{tf} \\ \text{Constraints} \quad & ((11)a) - ((11)s) \end{aligned} \quad (14)$$

where C_i is the estimated cost for instance i . The expression $(1 - \sum_{t \in L} z_{it})$ equals one if instance i is misclassified; otherwise, it equals zero. By incorporating these considerations, CSOCT-H provides a flexible framework to optimize classification performance while taking into account class imbalance and financial misclassification costs.

3.3. Optimal classification tree with hyperplane splits based on maximizing F1-score

In binary classification problems, the F1-score is a key metric used to evaluate classifier performance, especially in imbalanced datasets. It is based on the recall rates of the classifier for different classes and measures the classifier's ability to identify both the majority and minority classes by calculating the harmonic mean of their recall rates, as follows:

$$\text{F1-score} = \frac{2 \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2(n^+ - \text{FN})}{2n^+ - \text{FN} + \text{FP}} \quad (15)$$

The expression (15) can be rewritten in terms of the variable z_{it} as shown below:

$$\text{F1-score} = \frac{2 \sum_{i \in L} \sum_{i \in I: y_i=1} z_{it}}{n + \sum_{i \in L} \sum_{i \in I: y_i=1} z_{it} - \sum_{i \in L} \sum_{i \in I: y_i=0} z_{it}} \quad (16)$$

Then, the formulation of OCT-H that maximizes the F1-score can be designed as the following mixed-integer nonlinear model:

$$\begin{aligned} \max \quad & \frac{2 \sum_{i \in L} \sum_{i \in I: y_i=1} z_{it}}{n + \sum_{i \in L} \sum_{i \in I: y_i=1} z_{it} - \sum_{i \in L} \sum_{i \in I: y_i=0} z_{it}} - \alpha \sum_{f \in F} \sum_{t \in B} s_{tf} \\ \text{Constraints} \quad & ((11)a) - ((11)s) \end{aligned} \quad (17)$$

Given that the F1-score (refer to Eq. (16)) is a nonlinear function that cannot be directly processed by most solvers, this paper converts model (17) into the following mixed-integer quadratically constrained programming (MIQCP) model:

(OCT-H-F1):

$$\begin{aligned} \max \quad & f_1 - \alpha \sum_{f \in F} \sum_{t \in B} s_{tf} \\ \text{Constraints} \quad & ((11)a) - ((11)s) \\ & f_1(n + \sum_{t \in L} \sum_{i \in I: y_i=1} z_{it} - \sum_{t \in L} \sum_{i \in I: y_i=0} z_{it}) \leq 2 \sum_{t \in L} \sum_{i \in I: y_i=1} z_{it} \end{aligned} \quad (18)$$

where f_1 is a decision variable. By combining the objective of maximizing f_1 with the second constraint in model (18), f_1 is set equal to the value of the F1-score on the training dataset. In this paper, the solver Gurobi (version 11) (Gurobi Optimization, 2024) is used to solve this model.

4. Scalable methods

In this section, we first introduce a data sample size reduction method based on data binning and feature selection. Following this, we propose two solution methods to solve the large datasets. The first is a heuristic approach similar to the greedy method used in CART, and the second utilizes warm-start techniques to speed up the solving process.

4.1. Reduce the data sample size

From a computational perspective, model (11) and its variants exhibit two main shortcomings. First, when dealing with continuous features that have extreme values, normalizing these features to the $[0, 1]$ range can lead to most normalized values clustering near 0. This clustering can diminish the discriminative power of these features, especially due to the constraint $\mu = 0.005$ in ((11)o), thereby potentially degrading the model's performance. Second, finding the optimal classification tree is an NP-hard problem (Laurent & Rivest, 1976). The number of binary variables involved is approximately $\mathcal{O}(2^D(|I| + |F|))$, which increases linearly with the number of samples $|I|$ and features $|F|$, and exponentially with the tree depth D . Consequently, model (11) encounters intractability issues as the sample size and/or number of features grow.

To address the first issue, we apply the supervised discretization algorithm based on the Minimum Description Length Principle (MDLP) proposed by Fayyad and Irani (1993). This algorithm converts each continuous feature into a categorical feature by evaluating its statistical significance with respect to the class labels. Subsequently, we use one-hot encoding to transform each categorical value into a new binary column that indicates the presence or absence of the category with 1 s and 0 s. An interesting observation in the discretized datasets is that many samples share identical labels and features. For instance, in the UK dataset, the training set comprises 22,500 samples, yet it contains only 4586 unique samples. This implies that by utilizing the unique dataset of the UK, the number of variables in the OCT-H model is reduced to one-fifth of that in the original dataset, thereby accelerating the model's solution process. The upper bound on the number of unique samples in a binary dataset is determined by the following inequality:

$$|U| \leq \min\{2^{(|F'|+1)}, |I|\} \quad (19)$$

where $U = \{(\mathbf{x}_i, y_i)\}_{i=1}^{|U|}$ is the unique dataset of the original dataset I , $|U|$ is the number of unique samples, and $|F'|$ is the number of binary features after discretization.

Let w_i represent the frequency of the unique instance i in the original dataset I . The relationship between the total number of instances $|I|$ and the number of unique instances $|U|$ is given by the following equation:

$$\sum_{i \in U} w_i = |I| \quad (20)$$

Therefore, according to (20), we can give the equivalent formulation of (18) based on the unique dataset, which is as follows:

$$\max \quad f_1 - \alpha \sum_{f \in F} \sum_{t \in B} s_{tf}$$

Constraints ((11)a)–((11)l)

$$z_{it} \leq c_{tk}, \quad k = y_i, \forall i \in I \quad ((11)m)$$

$$z_{it} \leq l_t, \forall i \in U, \quad \forall t \in L \quad ((21)n)$$

$$\sum_{i \in U} w_i z_{it} \geq S_{\min} l_t, \quad \forall t \in L \quad ((21)o)$$

$$\begin{aligned} \sum_{f \in F'} x_{if} a_{jf} &\geq b_j - 2 \left(1 - \sum_{\forall i \in L_R(j)} z_{it} \right), \\ \forall i \in U, \forall j \in B, \end{aligned} \quad ((21)p) \quad (21)$$

$$\begin{aligned} \sum_{f \in F'} x_{if} a_{jf} + \mu d_j &\leq b_j + (2 + \mu) \left(1 - \sum_{\forall i \in L_L(j)} z_{it} \right), \\ \forall i \in U, \forall j \in B \end{aligned} \quad ((21)q)$$

$$\begin{aligned} f_1(n + \sum_{t \in L} \sum_{i \in U: y_i=1} w_i z_{it} - \sum_{t \in L} \sum_{i \in U: y_i=0} w_i z_{it}) \\ \leq 2 \sum_{t \in L} \sum_{i \in U: y_i=1} w_i z_{it}, \end{aligned} \quad ((21)r)$$

$$b_t \in [0, 1], \quad \forall t \in B, \forall f \in F' \quad ((21)s)$$

$$a_{if}, c_{tk}, z_{it}, l_t, d_t \in \{0, 1\}, \quad \forall i \in U, \quad \forall t \in L, \forall k \in K \quad ((21)t)$$

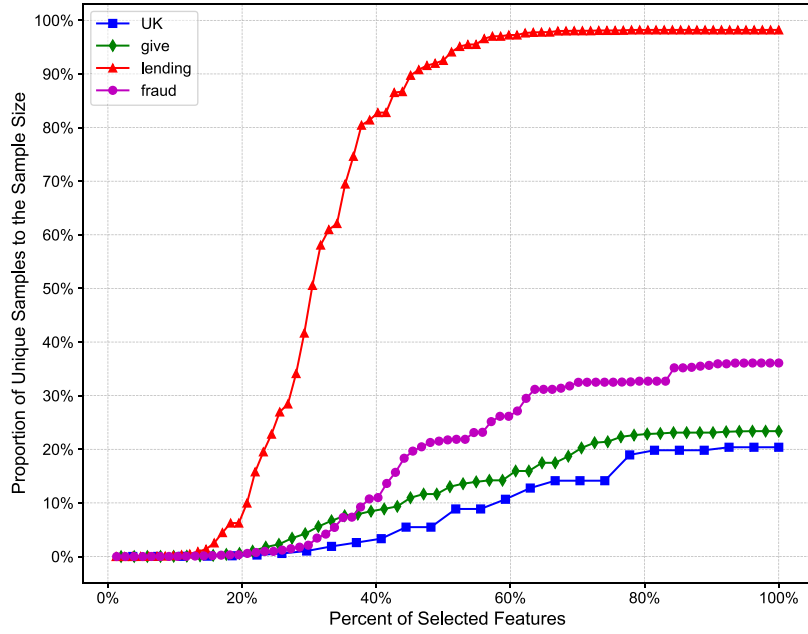


Fig. 2. The proportion of unique samples to the original samples under different percent of cumulative feature importance.

Model (21) can significantly reduce the solving time compared to model (18) when the number of unique samples is less than the size of the training dataset. For the UK dataset, where $|I| = 22,500$ and $|U| = 4586$, the number of binary variables in model (21) is one-fifth of that in model (18). This reduction is crucial because the complexity of solving NP-hard problems typically grows exponentially with respect to the number of decision variables. Consequently, reducing the problem size to one-fifth of the original can result in a solving time that is significantly less than one-fifth of the original time. The equivalent formulations of models (11) and (13) based on the unique dataset can be similarly designed by simply replacing the objective function of model (21).

According to Eq. (19), when the dataset I contains fewer features, it is more likely to obtain a unique dataset U with a reduced number of samples. In this study, we employ the random forest (RF) (Breiman, 2001) to select features based on their importance, aiming to decrease the unique sample size. Subsequently, we apply the models to the unique dataset to reduce solution time.

Fig. 2 illustrates the proportion of unique sample size to the original sample size across different percentages of selected features for four datasets (refer to Table 6 for more details about these datasets). Specifically, when all features are selected, the unique sample size for the give and UK datasets accounts for approximately 20% of their original sample size. For the Lending dataset, the unique sample size is about 35% of the original, while for the fraud dataset, it is nearly equal to the original sample size. However, when selecting only the top 20% of important features, the proportion of unique samples in these datasets drops to less than 10% of the original sample size.

4.2. The heuristic solution method for OCT-H

Although the sample size reduction strategies discussed in Section 4.1 can expedite the solution process, obtaining a near-optimal solution for model (21) with a depth greater than 2 remains very time-consuming using existing solvers. To address this challenge, we propose a heuristic solution method to effectively solve model (21).

4.2.1. A special formulation of OCT-H with $D = 1$

When the depth of tree is equal to 1 ($D = 1$), model (21) can be equivalently written as a MIP model with less decision variables as

follows:

$$\begin{aligned}
 \max \quad & J = f_1 - \alpha \sum_{f \in F} a_f \\
 \sum_{f \in F} \hat{a}_f & \leq 1, & ((22)a) \\
 a_f & \leq d_t, \quad \forall f \in F', \forall t \in B & ((22)b) \\
 \sum_{f \in F'} x_{if} a_f & \geq b - 2(1 - p_i), \quad \forall i \in U & ((22)c) \\
 \sum_{f \in F'} x_{if} a_f + \mu & \leq b + 2p_i, \quad \forall i \in U & ((22)d) \\
 \sum_{f \in F'} a_f & \geq 1, & ((22)e) \\
 \sum_{f \in F'} a_f & \leq N, & ((22)f) \\
 \sum_{i \in I} p_i & \geq S_{\min}, & ((22)g) \\
 \sum_{i \in I} (1 - p_i) & \geq S_{\min}, & ((22)h) \\
 f_1(n + \sum_{i \in U: y_i=1} w_i p_i - \sum_{i \in U: y_i=0} w_i p_i) & \leq 2 \sum_{i \in U: y_i=1} w_i p_i, & ((22)i) \\
 b & \in [-1, 1], & ((22)j) \\
 a_f, p_i & \in \{0, 1\}, \quad \forall i \in I, & ((22)k)
 \end{aligned} \tag{22}$$

For simplification, we denote the aforementioned linear binary classification model (22) as the LBC-F1 model. Let p_i be a binary decision variable, where $p_i = 1$ if instance i is predicted as 1, and $p_i = 0$ otherwise. For $D = 1$, model (21) requires $4|U| + |F| + 7$ binary variables and $2|F| + 1$ continuous decision variables. In contrast, model (22) only requires $|U| + |F|$ binary variables and $2|F| + 1$ continuous decision variables.

To demonstrate the effectiveness of the model, we use the UK dataset (see Section 5) for comparisons. With a time limit set to 600 s and $\alpha = 0$, the results obtained using Gurobi (Gurobi Optimization, 2024) are presented in Table 4. We compare the effectiveness of the two methods (LBC-F1 and OCT-H-F1) using metrics such as optimality gap, objective value (ObjVal), upper bound (UB), and running time.

- UB: The upper bound is the highest value that the objective function can take for the given problem. It serves as a benchmark to evaluate the performance of the model.

Table 4
Comparison of computational speed.

N	LBC-F1				OCT-H-F1			
	Gap	ObjVal	UB	Runtime (s)	Gap	ObjVal	UB	Runtime (s)
1	0	0.1618	0.1618	2.968	0.00%	0.1618	0.1618	262.557
2	0	0.3743	0.3743	32.418	156.31%	0.3743	0.9595	600.543
3	0	0.4296	0.4296	20.948	122.38%	0.4296	0.9554	601.219
4	0	0.4943	0.4943	18.419	88.29%	0.4943	0.9307	601.426
5	0	0.5135	0.5135	27.645	86.88%	0.5135	0.9596	600.564
6	0	0.5211	0.5211	56.029	79.21%	0.5211	0.9339	600.923
7	0	0.5211	0.5211	330.465	86.86%	0.5135	0.9595	601.033
8	0	0.5211	0.5211	530.967	118.34%	0.4395	0.9596	601.624
9	0	0.5211	0.5211	494.81	84.89%	0.5211	0.9636	606.44
10	0	0.5211	0.5211	542.531	85.70%	0.5211	0.9677	602.069

- **ObjVal:** This is the value of the objective function for the solution found by the model. It represents the quality of the solution in terms of the objective being optimized. In the maximization problem, ObjVal equals the lower bound (LB). For models LBC-F1 and OCT-H-F1, the ObjVal equals the F1-scores of these two classifiers on the training dataset.
- **Gap:** The gap represents the difference between the best-known solution and the UB, is calculated as $\frac{UB-ObjVal}{ObjVal} \times 100\%$. It measures the UB closeness from the best-searched solution (i.e. UB) to its possible global optimum (i.e. ObjVal). A gap of 0% means the solution is optimal.
- **Runtime:** This is the amount of time taken by the model to reach the solution. It indicates the computational efficiency of the model.

LBC-F1 consistently achieves an optimality gap of 0% across all datasets, indicating that it always finds the optimal solution within 600 s. In contrast, OCT-H-F1 fails to find and prove its objective value as optimal within the same time frame when N is greater than 1. Specifically, when $N = 1$, LBC-F1 requires only 3 s to find the optimal solution, whereas OCT-H-F1 takes 262 s.

Although both models yield identical objective values (ObjVal) for 8 out of 10 cases, the ObjVal of OCT-H-F1 is significantly lower than that of LBC-F1 in the cases when N is 7 and 8. For instance, when $N = 8$, the ObjVal of LBC-F1 is 0.5211, which is 0.0816 higher than that of OCT-H-F1. This indicates that the F1-score of LBC-F1 on the UK dataset is 0.0816 higher than that of OCT-H-F1.

For OCT-H-F1, the upper bound (UB) is significantly higher than the ObjVal (LB), suggesting that the model has not yet converged to the optimal solution and there is room for improvement. Fig. 3 illustrates the LB and UB over time when $N = 5$. LBC-F1 can find the optimal value within 20 s and requires only about 7 s to prove its optimality, whereas OCT-H-F1 takes more than 250 s to find the optimal value and spends more than 3000 s to prove its optimality.

In summary, LBC-F1 demonstrates superior performance in both solution quality and computational efficiency. It consistently finds the optimal solution with an optimality gap of 0% and does so in a shorter runtime. In contrast, OCT-H-F1, while producing comparable objective values (ObjVal) in most cases, often fails to find the optimal solution and requires significantly more computational time. These findings suggest that LBC-F1 is a more reliable and efficient choice for solving the OCT-H-F1 model with a depth of 1.

4.2.2. The heuristic algorithm

In this article, we propose a heuristic method based on the LBC model, which is similar to the recursive CART model (Breiman et al., 1984). There are three main differences. First, at each branch node, the RF is used to select the top `number_important_features` important features. Second, we employ a hyperplane partitioning strategy. Third, the criteria for sample splitting are based on specific objectives, such as the F1-score and accuracy, rather than traditional metrics like the Gini coefficient or information gain.

For example, when maximizing the F1-score, the details are presented in Algorithm 1. The subset assigned to node t is denoted as $(\mathbf{x}^t, \mathbf{y}^t)$. At branch node 1, LBC-F1 is applied to partition the dataset into two subsets. If the sample size of a subset is less than the predetermined `min_samples_split` or if the samples in the subset have the same labels, then partitioning of this subset stops. Otherwise, the LBC-F1 model continues to partition the subset. This process is repeated until a decision tree of predetermined depth is generated or the stopping conditions are satisfied.

Algorithm 1 The heuristic solution method

```

1: Input:  $S = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^{|\mathcal{I}|}$ ,  $B = \{1, 2, \dots, 2^D - 1\}$ ,  $L = \{2^D, \dots, 2^{D+1} - 1\}$ , min_samples_split
2: Output: Locally optimal classification tree  $\mathbb{T}$ 
3: for  $t = 1$  to  $|B|$  do
4:   if  $|\mathbf{y}^t| \geq \text{min\_samples\_split}$  AND the labels of dataset  $(\mathbf{x}^t, \mathbf{y}^t)$  are not the same then
5:     Random forest is used to select number_important_features top important features
6:     Generate  $U = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^{|\mathcal{U}|}$  according to the selected features
7:      $\mathbf{a}_t, \mathbf{b}_t \leftarrow LBC(\mathbf{x}^t, \mathbf{y}^t)$ 
8:   else
9:      $t$  considered as a leaf node
10:  end if
11: end for
12: return  $\mathbb{T}$ 

```

When applying Algorithm 1, the depth D , the number of features used at a branch node N , the minimum number of instances at each node (`min_samples_split`), and the important features selected by random forest (`number_important_features`) are the main parameters that affect the accuracy and complexity of the model. After the tree is generated by Algorithm 1, an important step is to reduce overfitting in classification trees and improve their generalization ability through post-pruning.

The post-pruning method follows the same steps as the cost-complexity pruning method (Bradford, Kunz, Kohavi, Brunk, & Brodley, 1998), but with a different cost function defined as follows:

$$C_\alpha(\mathbb{T}) = 1 - \text{F1-score}(\mathbb{T}) + \alpha \cdot |\mathbb{T}| \quad (23)$$

where α is the complexity parameter and $|\mathbb{T}|$ is the number of features used in the tree $\mathbb{T}$.

To illustrate the steps of this heuristic algorithm, we use the UK dataset (details of this dataset are presented in Table 6) to demonstrate its process. We set `min_samples_split` to 100, N to 5, and `number_important_features` to 20. At branch node 1, we apply the LBC-F1 model, resulting in a tree with a depth of 1, where $b_1 = -0.1069$ and $\mathbf{a}_1 = [0.006, 0, 0, 0, 0, -0.005, 0, -0.005, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, -0.982, 0, 0, 0, 0, 0]$. Repeating this process, we obtain a

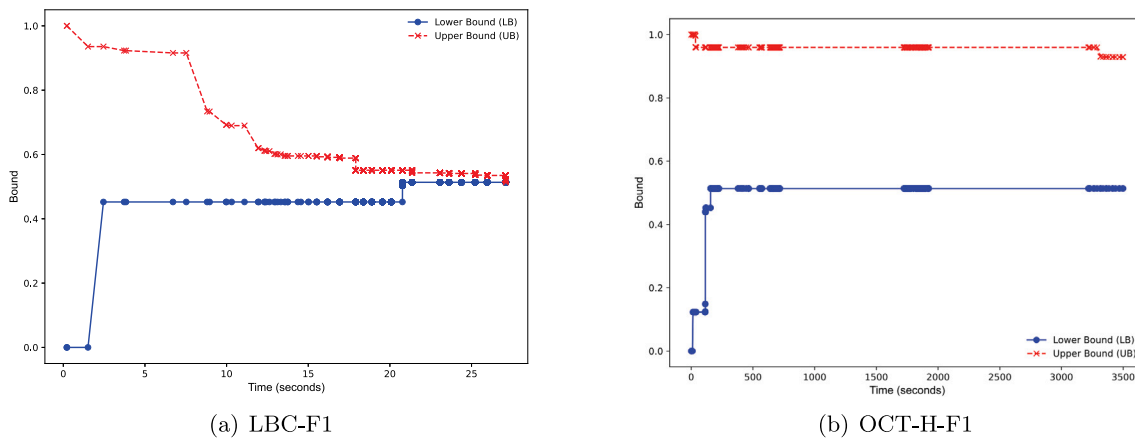


Fig. 3. Comparison of upper and lower bound evolution for learning the optimal classification tree with a depth of 1 on the UK dataset with 30000 samples when $N = 5$.

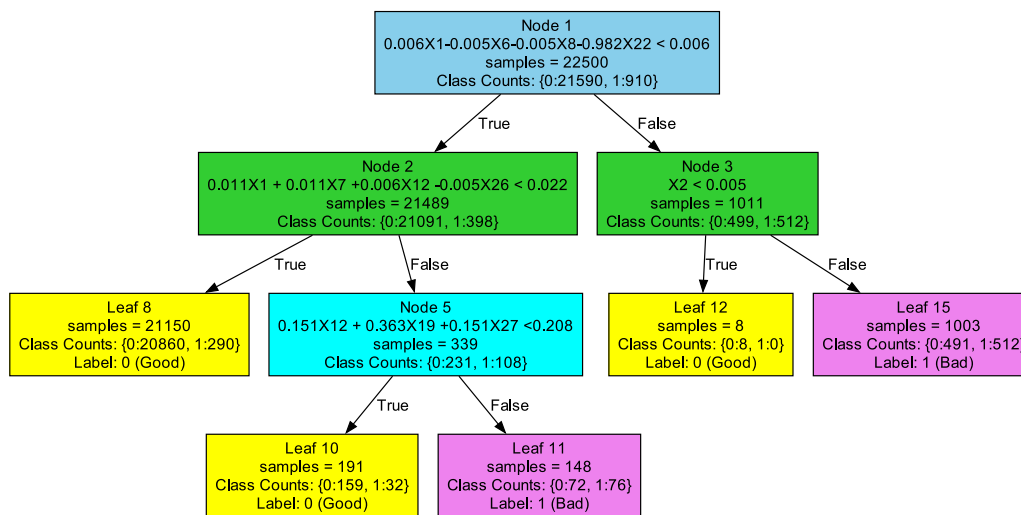


Fig. 4. The classification tree of the UK dataset by Algorithm 1.

classification tree with a depth of 3, as shown in Fig. 4. The F1-score on the training set of UK is 0.5706.

For the post-pruning process, we set α to 0.005. According to Eq. (23), we determine that branch node 3 should be pruned and replaced by a leaf node labeled “Bad”, because the F1-score of this pruned tree (refer to Fig. 5) is 0.5684, which is only 0.0022 less than that of the original tree.

4.3. The warm start method

Providing an integer-feasible solution as a warm start can significantly enhance the performance of MIP solvers (Bertsimas & Dunn, 2017; Bertsimas & Weismantel, 2005). This warm start offers a solid starting lower bound for the optimal solution, enabling more efficient pruning of the search tree and serving as a foundation for local search heuristics. The advantages gained from a warm start increase with its quality, making it beneficial to quickly and heuristically identify a strong integer-feasible solution prior to solving.

To illustrate the impact of warm starts, consider the scenario where $N = 5$ and $\alpha = 0$. Fig. 6 depicts the progression of upper and lower bounds while learning an optimal classification tree with a depth of 1 (LBC-F1 model) on the UK dataset, which contains 30,000 samples. Without a warm start, the LB gradually rises until the optimal solution is reached after 25 s, with an additional 30 s needed to confirm its optimality.

By injecting the CART solution) as a warm start, the same optimal solution is achieved in just 7 s, with only 13 more seconds required to verify its optimality. In this example, the total time to find and confirm the optimal solution is reduced by a factor of 3.6 with the warm start, and the time to find the optimal solution alone is reduced by approximately 2.3 times. This demonstrates that high-quality warm start solutions can significantly reduce MIP solution time.

For learning the OCT-H-F1 with a depth greater than 1, we use the solution generated by LBC-F1 for depth 1 as the warm start for depth 2. Similarly, the solution for depth D is used as the warm start for depth $D + 1$. The comparison results of cold start and warm start for the OCT-H-F1 on the UK dataset are presented in Table 5. As the depth D increases, the results from the cold start are significantly poorer compared to the warm start strategy, especially when $D \geq 3$.

However, the OCT-H-F1 with the warm start strategy cannot prove optimality within 600 s. We observe that the majority of the time is spent proving that the solution is optimal. Therefore, it is reasonable to terminate the solver early once the best solution has not changed for a certain period, as this typically indicates that the solution is indeed optimal or close to it, and proving optimality may require significantly more time.

5. Experimental comparison

In the experiments, we compare the proposed method with commonly used machine learning methods in credit scoring, including

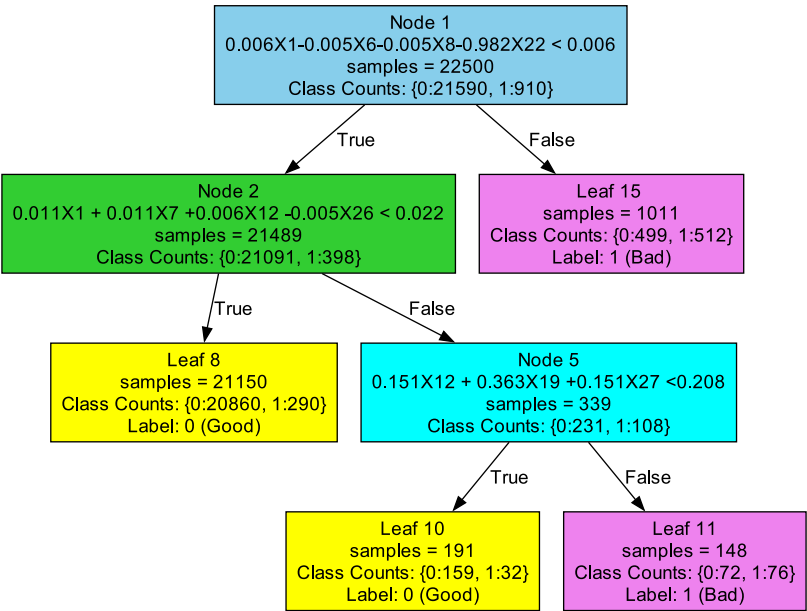


Fig. 5. The pruned classification tree of the UK dataset when $\alpha = 0.005$.

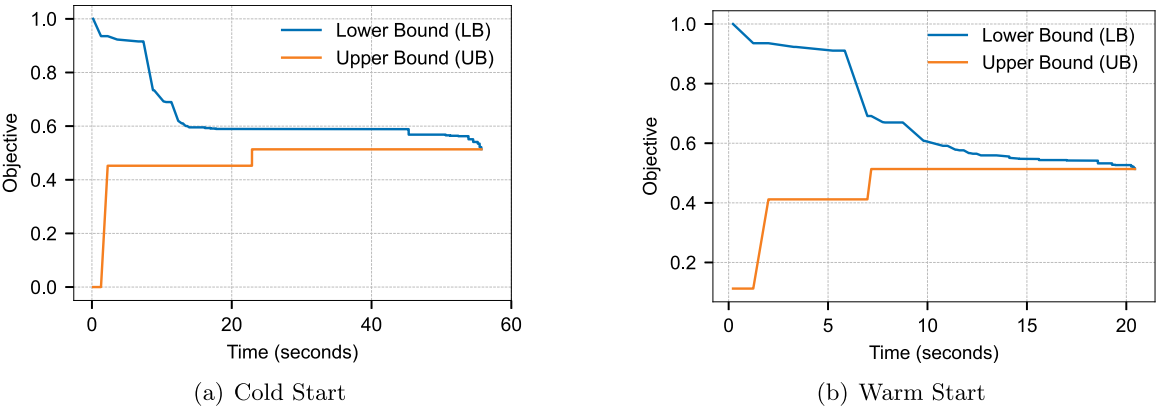


Fig. 6. Comparison of upper and lower bound evolution for learning the optimal classification tree with a depth of 1 on the UK dataset with 30000 samples when $N = 5$.

Table 5 Comparison of Cold Start and Warm Start Results.									
Depth	N	Cold start				Warm start			
		Gap (%)	ObjVal	UB	Runtime (s)	Gap (%)	ObjVal	UB	Runtime (s)
1	1	0.00	0.3743	0.3743	317.974	0.00	0.3743	0.3743	198.258
	2	91.31	0.5181	0.9913	600.399	93.00	0.5181	1.0000	600.456
	3	91.75	0.5215	1.0000	601.136	86.55	0.5360	1.0000	601.319
	4	92.51	0.5194	0.9998	601.382	86.55	0.5360	1.0000	601.153
	5	92.98	0.5181	0.9999	600.830	86.55	0.5360	1.0000	601.108
2	1	142.78	0.4117	0.9995	600.447	101.91	0.4943	0.9980	601.245
	2	85.98	0.5377	1.0000	602.083	85.98	0.5377	1.0000	603.014
	3	95.87	0.5105	1.0000	601.272	86.55	0.5360	1.0000	603.509
	4	88.69	0.5300	1.0000	603.128	85.98	0.5377	1.0000	600.674
	5	94.75	0.5135	1.0000	602.086	84.76	0.5413	1.0000	600.535
3	1	1379.18	0.0676	1.0000	635.237	94.94	0.5130	1.0000	621.232
	2	116.67	0.4615	1.0000	669.402	93.55	0.5167	1.0000	611.844
	3	802.13	0.1108	1.0000	627.143	82.99	0.5465	1.0000	604.355
	4	107.99	0.4808	1.0000	680.152	93.55	0.5167	1.0000	610.307
	5	96.86	0.5080	1.0000	643.054	84.35	0.5424	1.0000	615.070
4	1	1240.47	0.0746	1.0000	600.894	92.00	0.5208	1.0000	646.514
	2	951.81	0.0951	1.0000	600.817	93.55	0.5167	1.0000	600.743
	3	1673.43	0.0564	1.0000	635.042	82.99	0.5465	1.0000	613.021
	4	457.47	0.1794	1.0000	633.480	93.55	0.5167	1.0000	615.149
	5	1035.95	0.0880	1.0000	625.283	84.35	0.5424	1.0000	613.521

Table 6
Details of the four credit datasets.

Name	Cases	Features	Binarized features	Good: Bad	Source
UK	30,000	14	27	24.00:1	Baensens et al. (2003)
Give	120,271	10	51	13.49:1	Kaggle
Lending Club	160,000	19	82	4.35:1	Lending Club
Fraud	776,668	34	123	63.91:1	Kaggle

Logistic Regression (LR), CART Decision Tree (DT) (Breiman et al., 1984), Random Forests (RF) (Breiman, 2001), LightGBM (Ke et al., 2017), and XGBoost (Chen & Guestrin, 2016).

5.1. Experimental setting

Dataset: Our computational experiments are based on four public datasets for credit scoring: UK provided by Baensens et al. (2003), Give,¹ Lending Club,² and Fraud.³ For the Lending Club dataset, we randomly selected 40,000 samples labeled as “Fully Paid” and “Charged Off” per year from 2015 to 2018. We treated loans labeled as “Fully Paid” as “Good” and those labeled as “Charged Off” as “Bad”. The MDLP (Fayyad & Irani, 1993) method is applied to convert each continuous feature into a categorical feature based on its statistical significance for the class.

Model training: We randomly divided the datasets (UK, Give, and Fraud) into two subsets: a training dataset (75%) and a test dataset (25%). The Lending Club dataset was split according to the date, with loans from 2015, 2016, and 2017 forming the training set (75%) and loans from 2018 forming the test dataset (25%).

To select the optimal parameter combination, we performed four-fold cross-validation within the training set, where the training set was further partitioned into four equal-sized subsets. In each iteration, one subset was used as the validation set, while the remaining three subsets were used for training. This process was repeated four times, allowing each subset to serve as the validation set once. During each cross-validation iteration, grid search was employed to explore a pre-defined parameter space, as shown in Table 7. The models (DT, LR, RF, XGBoost, and LightGBM) were trained with different parameter combinations, and their performance was evaluated based on the Area Under the Curve (AUC) metric on the validation set. The parameter combination that yielded the highest AUC across the cross-validation folds was selected as the optimal set of parameters. Finally, the model trained with the optimal parameters on the entire training set was validated on the test set to assess its generalization performance.

The algorithms (DT, RF, LR) are implemented in Python 3.7 using scikit-learn (Pedregosa et al., 2011). LightGBM and XGBoost are also implemented in Python 3.7. The OCT-H models including OCT-H-Acc, OCT-H-F1 and CSOCT-H are solved by Gurobi (version 11.1) (Gurobi Optimization, 2024), with a time limit of 30 min for all datasets. All the tests are conducted on 13th Gen Intel(R) Core(TM) i9-13900KF 3.00 GHz and a memory of 32 GB. The codes (OCT-H-Acc, CSOCT-H and OCT-H-F1) are available at GitHub.⁴ The implementation code and corresponding parameter documentation for other models will be uploaded to GitHub later.

This paper divides OCT-H models into two main models based on solving methods: the heuristic method (refer to Section 4.2.2) and global optimization using warm start (refer to Section 4.3). The detailed illustrations of the parameters in OCT-H (Heuristic) are as follows:

- **max_depth:** This parameter sets the maximum depth of the tree. A deeper tree can capture more complex patterns but may also lead to overfitting and increase computation time. The default value is 2.
- **α :** This parameter controls the complexity of the tree. A larger value of α results in a sparser tree, which can help prevent overfitting. The default value is 0.01.
- **N:** This is the maximum number of features that can be used at a branch node. It also influences the complexity of the tree. The default value is 5.
- **min_samples_split:** This parameter specifies the minimum number of instances required at a branch node to consider splitting. The default value is 20.
- **objective:** This defines the objective of the OCT-H model. **objective = accuracy:** The model aims to maximize accuracy (OCT-H-Acc). **objective = F1-score:** The model aims to maximize the F1-score (OCT-H-F1). **objective = cost-sensitive:** This is for the cost-sensitive OCT-H (CSOCT-H). For the credit scoring problem, the default value is **objective = F1-score**.
- **C_{FN} :** This parameter only functions when **objective = cost-sensitive** and is to address class imbalance. The default value is the ratio of goods to bads.
- **number_important_features:** This parameter indicates the number of features selected by the Random Forest (RF) at each branch node. The default value is 15.

In the OCT-H (Optimal) model, the parameters generally have the same meanings as those in the OCT-H (Heuristic), with one key difference regarding the parameter **number_important_features**. **number_important_features** in OCT-H (Optimal) is the total number of features selected by the RF, thus, the total number of features used by OCT-H (Optimal) cannot exceed **number_important_features**, the default value is 15.

5.2. Evaluation metrics

Financial institutions use credit scoring models to estimate and manage the risk associated with lending decisions, aiming to minimize potential losses due to defaults and to make objective and reliable choices. A credit score represents an estimation of the Probability of Default (PD) for a borrower, which reflects the likelihood that a consumer will fail to fulfill their financial obligations if a loan is granted, based on historical data and individual characteristics.

Formally, for a given applicant i , the PD is estimated as $\hat{p}_i = P(y_i = 1 \mid \mathbf{x}_i)$, where $\mathbf{x}_i$ represents the features or attributes of applicant i , $y_i = 1$ indicates that the applicant defaults on the loan and $y_i = 0$ indicates that the applicant repays the loan as agreed. Logistic Regression (LR), LightGBM, and XGBoost can directly provide predicted PDs $\hat{p}_i$ for each applicant. For Decision Trees (DT) and OCT-H, the PD for an applicant is estimated as the proportion of bad applicants in the leaf node reached by traversing the tree from root to leaf based on the applicant's features. For Random Forests (RF), the PD is calculated by averaging the predicted PDs across all trees in the ensemble:

$$\hat{p}_i = \frac{1}{T} \sum_{t=1}^T \hat{p}_i^{(t)},$$

where T is the total number of trees, and $\hat{p}_i^{(t)}$ is the predicted PD from the t th tree.

¹ <https://kaggle.com/competitions/GiveMeSomeCredit>

² <https://www.lendingclub.com/>

³ <https://www.kaggle.com/datasets/ranjeetshrivastav/fraud-detection-dataset>

⁴ <https://github.com/Tommytutu/OCT-H>

Table 7
Parameter settings for the selected models.

Algorithm	Parameter	Range	Stride
OCT-H (Heuristic)	max_depth	[1, 5]	1
	α : the complexity parameter	{0.001, 0.01, 0.1}	
	min_samples_split	[5, 100]	20
	N	[1, 10]	2
	objective	{F1-score, accuracy, cost-sensitive}	
OCT-H (Optimal)	number_important_features	[5, 20]	5
	max_depth	[1, 5]	1
	α : the complexity parameter	{0.001, 0.01, 0.1}	
	N	[1, 10]	2
	objective	{F1-score, accuracy, cost-sensitive}	
LR	number_important_features	[10, 25]	5
	C	{0.1, 1, 10, 100}	
	penalty	{L1, L2}	
DT	class_weight	{None, balanced}	
	criterion	{gini, entropy}	
	max_depth	[1, 10]	1
	min_samples_split	[10, 100]	10
	min_samples_leaf	[5, 100]	5
RF	class_weight	{None, balanced}	
	n_estimators	[100, 1000]	100
	maximum depth of each DT	[1, 10]	1
	min_samples_split	[10, 100]	10
	min_samples_leaf	[5, 100]	5
	class_weight	{None, balanced}	
	max_features	{log2, sqrt, None}	
LightGBM	n_estimators	[100, 1000]	100
	maximum depth of each DT	[1, 10]	1
	min_samples_split	[10, 100]	10
	min_samples_leaf	[5, 100]	5
	scale_pos_weight	{1, n^-/n^+ }	
	learning_rate	{0.01, 0.1, 0.2}	
	lambda_l1	{0, 0.1, 1}	
XGBoost	lambda_l2	{0, 0.1, 1}	
	n_estimators	[100, 1000]	100
	maximum depth of each DT	[1, 10]	1
	min_samples_split	[10, 100]	10
	min_samples_leaf	[5, 100]	5
	scale_pos_weight	{1, n^-/n^+ }	
	learning_rate	{0.01, 0.1, 0.2}	
	gamma	{0, 0.1, 0.2}	

We compare the predictive performance of the competing models using three criteria. Firstly, as is commonly used in the literature (see above references) and by practitioners, we also use metrics that require an account to be classified as either good or bad: accuracy and F1-score, which are computed from the confusion matrix at a fixed cut-off threshold. Secondly, we use two standard statistical measures, Receiver Operating Characteristic curves (ROC) and Brier Score (BS), which are commonly used in the credit scoring literature (Lessmann et al., 2015). Thirdly, we use model complexity to assess the complexity of interpretable models (DT, LR, and OCT-H).

5.2.1. Cut-off threshold-dependent metrics

The primary aim of credit scoring is to accurately estimate $\hat{p}_i$ to inform the decision of whether to grant a loan to applicant i . This decision is made by comparing the estimated PD to a cut-off threshold τ . The threshold is determined based on the lending institution's risk appetite and operational considerations. Since the actual cut-offs used by lenders are frequently not disclosed in academic research, we adhere to the methodology proposed by Rozo et al. (2023). We establish a cut-off for each model based on the proportion of observed good applicants in the training data. This approach ensures that the proportion of applicants predicted to be good matches the observed proportion, thus making the comparison independent of the individual models' threshold choices. Using the classifications derived from the fixed cut-off τ , we construct the confusion matrix for each model on the test set. From the confusion matrix, we compute the accuracy and F1-score. This approach allows for a fair comparison of different models under consistent classification

conditions and reflects practical lending scenarios where decisions are guided by estimated PDs and predefined risk thresholds. The steps are as follows:

1. Compute the proportion of good applicants. Let p_{Good} denote the proportion of Good applicants in the training dataset. Mathematically,

$$p_{\text{Good}} = \frac{\text{Number of Good Applicants in Training Data}}{\text{Total Number of Applicants in Training Data}}.$$

2. Determine the cut-off threshold. Rank all applicants in the training data in ascending order based on their predicted PDs. Identify the applicant at the percentile corresponding to p_{Good} . The predicted PD of this applicant is set as the cut-off threshold τ . Formally, if $\hat{p}_{(i)}$ represents the i th smallest predicted PD, then the cut-off τ is:

$$\tau = \hat{p}_{(k)}, \quad \text{where } k = \lceil |I| \times p_{\text{Good}} \rceil.$$

3. Apply the cut-off to the test data. Using the cut-off τ , classify applicants in the test set:

$$\hat{y}_i = \begin{cases} \text{Good (0),} & \text{if } \hat{p}_i < \tau, \\ \text{Bad (1),} & \text{if } \hat{p}_i \geq \tau, \end{cases}$$

where $\hat{p}_i$ is the predicted PD for the i th applicant in the test set.

McNemar's test: McNemar's Test is a non-parametric statistical method used to determine if there are significant differences between

Table 8
McNemar's test.

	Model B Correct	Model B Incorrect	Total
Model A Correct	a	b	$a + b$
Model A Incorrect	c	d	$c + d$
Total	$a + c$	$b + d$	n

the performances of two classification models on the same dataset (Dietterich, 1998). The McNemar's Test has been employed to compare and validate the effectiveness of different predictive models in credit scoring (West, 2000).

The test focuses on the instances where the two models disagree in their predictions. It examines whether the number of instances where model A is correct and model B is incorrect differs significantly from the number of instances where model B is correct and model A is incorrect. Consider a contingency table summarizing the paired predictions of the two models (see Table 8).

Under the null hypothesis that both models have the same error rate (i.e., any differences in predictions are due to chance), the test statistic is calculated as:

$$\chi^2 = \frac{(b - c)^2}{b + c},$$

which follows a chi-squared distribution with one degree of freedom.

Remark 3. While the fixed cut-off approach ensures that comparisons are made under consistent classification conditions by applying the same threshold τ across all models, determining the optimal value of τ can be particularly challenging, especially in academic research where real-world thresholds used by lenders are often proprietary or context-specific. Different choices of τ may lead to varying conclusions about model performance, potentially skewing results based on arbitrary or subjective criteria. This variability can undermine the fairness and validity of model comparisons. Therefore, using threshold-independent metrics such as the Area Under the Curve (AUC) provides a more reliable means of evaluation. AUC ensures that our comparisons are fair, robust, and aligned with both academic standards and practical considerations in credit scoring. By reflecting the model's ability to discriminate between classes across all possible thresholds, AUC mitigates the limitations associated with fixed cut-offs and captures the overall effectiveness of the predictive model in a more comprehensive manner. This makes it an invaluable tool for practitioners and researchers in the field.

5.2.2. Threshold-independent metrics

AUC: The area under the curve (AUC) is particularly valuable for evaluating classification models in binary contexts, representing the area under the ROC (Receiver Operating Characteristic) curve, which plots the True Positive Rate (TPR) against the False Positive Rate (FPR):

$$AUC = \sum_{i=1}^{n-1} (FPR_{i+1} - FPR_i) \cdot \frac{TPR_{i+1} + TPR_i}{2} \quad (24)$$

where FPR_i and TPR_i represent the False Positive Rate and True Positive Rate at the i th threshold, respectively.

The value of AUC ranges from 0 to 1, where an AUC of 0.5 indicates no discriminative ability (akin to random guessing), while an AUC of 1.0 denotes perfect classification. AUC provides a single scalar value that summarizes the model's ability to distinguish between positive and negative classes, making it an essential tool for model assessment.

However, simply calculating AUC is not always sufficient, especially when comparing multiple models. In such cases, understanding the variability of the AUC estimate becomes vital. This is where the DeLong method comes into play. The DeLong test is a non-parametric statistical method used to compare the areas under two correlated

Receiver Operating Characteristic (ROC) curves (AUCs) (DeLong, DeLong, & Clarke-Pearson, 1988). It is commonly applied when evaluating whether the difference between the AUCs of two diagnostic tests (or models) is statistically significant, especially when the tests are performed on the same set of subjects. The hypothesis test is given as follows:

H_0 : There is no difference between the AUCs of the two classifiers ($\theta_A = \theta_B$),

H_1 : There is a difference between the AUCs ($\theta_A \neq \theta_B$).

where θ_A is the AUC of model A and θ_B is the AUC of model B.

The test statistic Z is computed as:

$$Z = \frac{\theta_A - \theta_B}{\sqrt{\text{Var}(\theta_A - \theta_B)}} \quad (25)$$

Under H_0 (that there is no difference between the two AUCs), Z follows a standard normal distribution. Compute the p -value using the standard normal distribution:

$$p = 2 \times (1 - \Phi(|Z|)) \quad (26)$$

where Φ is the cumulative distribution function (CDF) of the standard normal distribution. A small p -value (typically $p < 0.05$) indicates that we can reject H_0 and conclude that there is a statistically significant difference between the two AUCs.

BS: The Brier Score (BS) describes the average probabilistic prediction error:

$$BS = \frac{1}{|I|} \sum_{i=1}^{|I|} (\hat{p}_i - \hat{y}_i)^2 \quad (27)$$

where $\hat{p}_i$ represents the predicted default probability of applicant i by the credit scoring model.

5.2.3. Complexity

Model complexity is used to measure the complexity of interpretable models (DT, LR, and OCT-H). For DT, complexity is defined as the number of leaf nodes. For LR and OCT-H, when applied to binarized datasets, the hyperplane can be transformed into a univariate tree (refer to Section 5.3.1). In this paper, LR performs better on binarized datasets than on continuous datasets. Consequently, the complexity of LR and OCT-H is equivalent to the number of leaf nodes in their corresponding univariate trees (see Figs. 7 and 8). Although interpretable machine learning models (DT, LR, and OCT-H) are transparent and interpretable, humans may struggle to understand models with many decision rules, as they can handle at most 7 ± 2 cognitive entities at once (Miller, 1956; Rudin, 2019). Therefore, complexity can be considered a useful measure of interpretability.

5.3. Results

In this section, we first compare our proposed models with the classic interpretable models (LR and DT); then compare with the tree-based ensemble models (RF, LightGBM, and XGBoost).

5.3.1. OCT-H vs. classical interpretable models

As mentioned earlier, all continuous features are binarized. When the hyperplane generated by the OCT-H is known, it can be reformulated as a univariate tree, making the OCT-H as interpretable as univariate trees like CART. For instance, the optimal OCT-H tree structure on the UK dataset is a tree with a hyperplane and a depth of 1, as illustrated in Fig. 7. Since the value of each feature is either 0 or 1, the solution space of the hyperplane

$$0.05X_1 + 0.01X_3 - 0.03X_5 + 0.015X_6 - 0.04X_7 + 0.005X_{11} + 0.03X_{22} + 0.01X_{26} < 0.0801$$

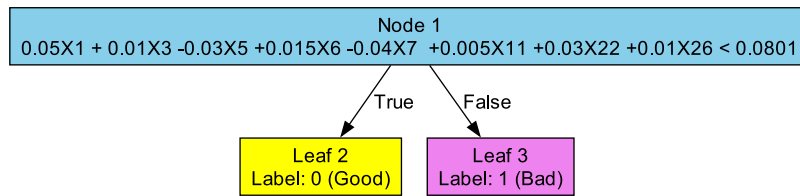


Fig. 7. OCT-H tree structure on the UK dataset.

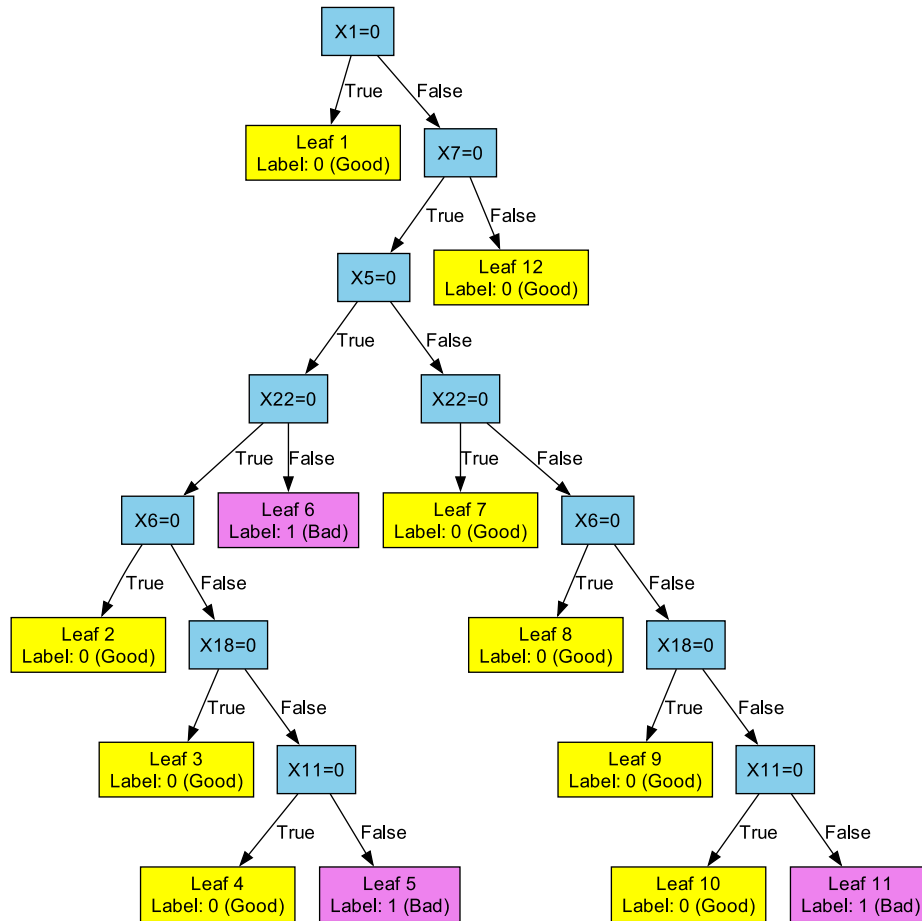


Fig. 8. The univariate classification tree corresponding to the OCT-H on the UK dataset.

is finite. Therefore, we can employ a brute force method to explore this space. Consequently, we can derive its corresponding univariate classification tree, as shown in Fig. 8.

In Fig. 8, there are 12 leaf nodes, with three nodes labeled as “Bad” and nine nodes labeled as “Good”. Each path from the root node to a leaf node represents a scoring rule. We use the proportion of bad applicants in the training dataset at each leaf node to represent the default probability for applicants that satisfy the scoring rule represented by that path on the test dataset.

For example, 7563 samples are assigned to leaf 2 of the tree in Fig. 8, and all of them are good applicants, so the credit default risk for applicants satisfying the rule represented by this node is 0. Leaf 5 has 1103 samples, with 451 good applicants and 652 bad applicants. Therefore, the corresponding default risk for this node is 0.5911 (652/1103). By the same process, we can calculate the default probability for each leaf node. By doing so, we can determine the probability of each sample. Finally, we use these probabilities to calculate the AUC and BS to evaluate the performance of the proposed models.

Similarly, LR can be converted into a univariate tree on the binary dataset. Thus, the complexity of these three models (LR, DT,

and OCT-H) is primarily determined by the decision rules they have, specifically the number of leaf nodes in their univariate classification trees. The fewer the number of leaf nodes (complexity), the stronger the interpretability of the model.

The results of the four different interpretable models across the four datasets are presented in Table 9, with the best value for each evaluation measure highlighted in bold. The DeLong test results for the AUCs and the McNemar’s test results are provided in Tables 10 and 11, respectively. Based on the findings from Table 9, we can draw the following conclusions:

1. Between OCT-H (Optimal) and OCT-H (Heuristic), the complexity of OCT-H (Optimal) is significantly lower than that of OCT-H (Heuristic) across all four datasets. On average, OCT-H (Optimal) achieves a complexity reduction of approximately 40%, thereby enhancing its interpretability. In terms of accuracy and F1-score, OCT-H (Optimal) exhibits slightly higher performance than OCT-H (Heuristic) across all datasets. Conversely, OCT-H (Heuristic) outperforms OCT-H (Optimal) in AUC on three out of the four

datasets. The discrepancies in performance between OCT-H (Optimal) and OCT-H (Heuristic) can largely be attributed to their respective problem-solving approaches. As OCT is recognized as an NP-hard problem, OCT-H (Optimal) cannot effectively solve deeper classification trees. In contrast, OCT-H (Heuristic) employs heuristic methods that enable it to learn deeper classification trees, consequently improving its fitting and predictive capabilities. However, this improved performance comes at the cost of increased model complexity.

- Both OCT-H (Optimal) and OCT-H (Heuristic) demonstrate superior performance compared to DT in terms of accuracy, F1-score, AUC, Brier Score (BS), and complexity across all datasets. The results of the DeLong test, as shown in Table 10, indicate that the AUCs of OCT-H are significantly higher than those of DT across all datasets. Furthermore, results from McNemar's test in Table 11 reveal that OCT-H exhibits higher accuracy than DT. In terms of model complexity, DT presents an average of 34.5 leaf nodes, suggesting a relatively high complexity level. In contrast, the OCT-H models have significantly fewer leaf nodes, averaging 6.25 and 10.5, respectively. This notable discrepancy arises from DT's reliance on a top-down greedy algorithm known as CART, which fails to learn optimal splits effectively.
- OCT-H (Optimal) and OCT-H (Heuristic) nearly consistently outperform Logistic Regression (LR) across all datasets regarding accuracy, F1-score, AUC, BS, and model complexity, as detailed in Table 9. However, two exceptions are noted. First, LR exhibits slightly higher accuracy on the UK dataset compared to OCT-H (0.9641 vs. 0.9635); however, this difference is not statistically significant, as indicated by McNemar's test in Table 11. Second, OCT-H (Optimal) achieves marginal improvements in F1-score (0.2976 vs. 0.2951) and AUC (0.7027 vs. 0.7017) over LR, yet these differences are minimal, with the DeLong test indicating that the improvement in AUC is not statistically significant (statistic = 1.991, $p = 0.051$). The AUC values across all models are relatively low in this dataset compared to other datasets, suggesting that the dataset poses greater challenges for predictive modeling, potentially due to temporal changes affecting out-of-time validation.

In summary, OCT-H demonstrates superior performance over LR and DT across multiple evaluation metrics and datasets. The statistically significant enhancements in AUC, as shown by the DeLong test, indicate that OCT-H possesses better discriminatory power than LR in the majority of datasets. Furthermore, reductions in BS suggest that OCT-H provides better-calibrated probability estimates. The lower complexity of OCT-H models enhances their interpretability, which is a critical factor in credit scoring applications. However, the Lending dataset serves as an exception, where the differences between OCT-H and LR are not statistically significant and AUC values are less impressive. This phenomenon may be a consequence of the out-of-time validation employed for this dataset, which could introduce temporal dynamics that are challenging for the models to capture adequately. The lower performance on this dataset highlights potential difficulties in generalizing the models to new time periods, underscoring an area that warrants further investigation.

5.3.2. OCT-H vs. tree-based ensemble models

According to Tables 9 and 12, tree-based ensemble models (RF, LightGBM, and XGBoost) significantly outperform classical machine learning methods (DT and LR) across all datasets. This indicates that tree-based ensemble models are highly effective in handling credit scoring problems involving imbalanced data, unlike existing single machine learning methods. These findings align with previous research conducted by Jiang et al. (2023) and Lessmann et al. (2015). Here, we primarily compare our proposed models with tree-based ensemble models (RF, LightGBM, and XGBoost). The results are presented in

Table 9

Performance Comparison of Different Interpretable Models on the Four Datasets.

		Accuracy	F1-Score	AUC	BS	Complexity
UK	DT	0.9589	0.5319	0.9444	0.0847	10
	LR	0.9641	0.5568	0.9455	0.0256	25
	OCT-H (Optimal)	0.9635	0.6130	0.9528	0.0251	7
	OCT-H (Heuristic)	0.9616	0.5932	0.9531	0.0251	10
Give	DT	0.9191	0.4201	0.8392	0.0530	45
	LR	0.9173	0.4068	0.8485	0.0526	51
	OCT-H (Optimal)	0.9215	0.4373	0.8526	0.0517	6
	OCT-H (Heuristic)	0.9212	0.4349	0.8532	0.0515	14
Lending	DT	0.7966	0.2666	0.6807	0.1225	62
	LR	0.8045	0.2951	0.7017	0.1181	82
	OCT-H (Optimal)	0.8052	0.2976	0.7027	0.1119	4
	OCT-H (Heuristic)	0.8044	0.2946	0.7026	0.1118	11
Fraud	DT	0.9712	0.0773	0.6790	0.1968	21
	LR	0.9715	0.0855	0.7513	0.0151	77
	OCT-H (Optimal)	0.9733	0.1750	0.7918	0.0211	8
	OCT-H (Heuristic)	0.9718	0.0958	0.7924	0.0151	9

Note: DT and LR are trained on both binary and original datasets, and we only report the best results on the test set. The AUC and F1 scores of the LR on all discrete datasets are better than those on continuous datasets.

Table 12. The DeLong test for the AUCs and the McNemar's test are presented in Tables 10 and 11, respectively. The detailed analysis of the results is as follows:

- In terms of accuracy, OCT-H outperforms traditional ensemble methods (RF, LightGBM, and XGBoost) on three datasets respectively. The McNemar's test further shows that OCT-H is statistically more accurate than RF on three datasets Give, Lending and Fraud (see in Table 11); OCT-H is statistically more accurate than LightGBM on the dataset Fraud, while there is no statistically significant difference between OCT and XGBoost across all the datasets.
- In terms of F1-score, the OCT-H models (both Optimal and Heuristic) consistently outperformed traditional ensemble methods (RF and LightGBM) across all datasets. OCT-H performs better on three datasets (UK, Give and Fraud). Notably, for dataset Fraud, OCT-H significantly improves the F1-score compared to XGBoost (from 0.0974 to 0.1750). This suggests that OCT-H models are more effective in accurately identifying both positive and negative instances, which is crucial in credit scoring scenarios where both false positives and false negatives can have significant implications. The consistent advantage of OCT-H models underscores their potential as a preferred choice for applications requiring high interpretability and balanced predictive performance.
- The AUC metric, which measures the ability of a model to distinguish between classes, provides insight into the overall discriminative power of the models. The AUC scores for OCT-H are greater than those of RF on three datasets (Give, Lending and Fraud), the DeLong test also shows that these differences on these three datasets are statistically significant. For the UK dataset, RF slightly performs better than OCT-H, but the differences are not statistically significant according to the DeLong test. LightGBM and XGBoost both outperform OCT-H in terms of AUC scores on three datasets, the differences are statistically significant on two datasets (Lending and Fraud). This indicates that LightGBM and XGBoost have a slight edge in distinguishing between classes.

Overall, the OCT-H model outperforms the RF model in both F1-score and AUC score. Additionally, OCT-H demonstrates comparable performance to LightGBM and XGBoost. While LightGBM and XGBoost exhibit slightly better AUC scores, they significantly lag behind OCT-H in terms of the F1-score. Notably, XGBoost outperforms OCT-H

Table 10
DeLong test estimates (p-values) for AUCs.

	UK	Give	Lending	Fraud
OCT-H vs. DT	2.988 (0.002 ^{***})	7.067 (0 ^{***})	13.570 (0 ^{***})	17.768 (0 ^{***})
OCT-H vs. LR	2.930 (0.003 ^{***})	8.093 (0 ^{***})	1.991 (0.051)	13.956 (0 ^{***})
OCT-H vs. RF	−0.358 (0.719)	0.195 (0.845)	1.944 (0.046 [*])	7.296 (0 ^{***})
OCT-H vs. XGBoost	−0.860 (0.389)	1.551 (0.120)	−4.499 (0 ^{***})	−2.995 (0.003 ^{**})
OCT-H vs. LightGBM	−1.653 (0.098)	0.589 (0.555)	−7.675 (0 ^{***})	−2.841 (0.005 ^{**})

Note: OCT-H denotes the best AUC of OCT-H (Optimal) and OCT-H (Heuristic). The values in parentheses represent the p-values from the DeLong test, which assess the statistical significance of differences in AUC scores between the listed models.

*** Significance levels are indicated as $p < 0.001$.

** Significance levels are indicated as $0.001 \leq p < 0.01$.

* Significance levels are indicated as $0.01 \leq p < 0.05$.

Table 11
McNamar's test estimates (p-values)

	UK	Give	Lending	Fraud
OCT-H vs. DT	4.083 (0.043 [*])	81.737(0 ^{***})	211.820 (0 ^{***})	552.003 (0 ^{***})
OCT-H vs. LR	2.920 (0.073)	93.159(0 ^{***})	1.694 (0.193)	16.28 (0 ^{***})
OCT-H vs. RF	1.205 (0.272)	15.058 (0.0001 ^{***})	10.498 (0.0012 ^{**})	12.1 (0.0005 ^{***})
OCT-H vs. XGBoost	2.876 (0.089)	1.804 (0.179)	2.02383 (0.155)	1.988 (0.159)
OCT-H vs. LightGBM	0 (1)	1.568 (0.210)	1.70163 (0.192)	7.111 (0.008 ^{**})

Note: OCT-H denotes the best AUC of OCT-H (Optimal) and OCT-H (Heuristic). The values in parentheses represent the p-values from the McNamar's test, which assess the statistical significance of differences between the listed models.

*** Significance levels are indicated as $p < 0.001$.

** Significance levels are indicated as $0.001 \leq p < 0.01$.

* Significance levels are indicated as $0.01 \leq p < 0.05$.

concerning both F1-score and AUC score on the Lending dataset. This difference can be attributed to the use of out-of-time validation specific to this dataset, as discussed in Section 5.3.1.

We acknowledge that Gradient Boosted Machines (GBM), particularly LightGBM, are recognized for their scalability and efficiency, even in the absence of GPU hardware. Although there exists a gap in solution efficiency and accuracy on large datasets between OCT-H and LightGBM, OCT-H can narrow this gap in predictive performance by establishing an appropriate objective function. To illustrate this, we conducted experiments where we set the F1-score, instead of AUC, as the evaluation metric during cross-validation for the tree-based ensemble models (RF, LightGBM, and XGBoost). We also configured the objective function of OCT-H to directly maximize the F1-Score. Under these conditions, our results indicate that OCT-H achieves higher F1-scores than RF across all four datasets and outperforms LightGBM and XGBoost in three out of four datasets. Furthermore, the AUC scores for OCT-H are statistically higher than those of RF in three datasets and exceed those of LightGBM and XGBoost in two datasets. This discrepancy arises because the training objective functions of tree-based ensemble models cannot directly maximize the F1-Score as OCT-H does.

In our study, we must note that AUC is independent of the cut-off threshold and is thus more robust than the threshold-dependent F1-score. Consequently, we opted for AUC as the evaluation metric when utilizing cross-validation to train our models. This choice ensures that the models are evaluated and compared on a threshold-independent metric, providing a fair and robust assessment of their performance. Inspired by these findings, we recognize that designing an OCT-H model that directly maximizes the AUC score could be a more effective approach for addressing credit scoring and similar classification tasks. This represents an interesting direction for future research, as such an approach has the potential to enhance both the predictive performance and the practical utility of OCT-H in real-world applications where AUC is a critical evaluation metric.

6. Conclusions

We introduce a new formulation of the optimal classification tree with hyperplane splits (OCT-H). This model is not only inherently

Table 12
Performance Comparison of OCT-H with Tree-based Ensemble Models.

		Accuracy	F1-score	AUC	BS
UK	RF	0.9663	0.5832	0.9532	0.0488
	XGBoost	0.9629	0.5775	0.9555	0.0245
	LightGBM	0.9673	0.5964	0.9567	0.0544
	OCT-H (Optimal)	0.9635	0.6130	0.9528	0.0251
	OCT-H (Heuristic)	0.9616	0.5932	0.9531	0.0251
Give	RF	0.9212	0.4349	0.8530	0.0518
	XGBoost	0.9202	0.4278	0.8511	0.0522
	LightGBM	0.9206	0.4311	0.8525	0.1452
	OCT-H (Optimal)	0.9215	0.4373	0.8526	0.0517
	OCT-H (Heuristic)	0.9212	0.4349	0.8532	0.0515
Lending	RF	0.8029	0.2893	0.7007	0.1188
	XGBoost	0.8063	0.3014	0.7078	0.1196
	LightGBM	0.8043	0.2942	0.7103	0.1188
	OCT-H (Optimal)	0.8052	0.2976	0.7027	0.1119
	OCT-H (Heuristic)	0.8044	0.2946	0.7026	0.1118
Fraud	RF	0.9717	0.0935	0.7785	0.1884
	XGBoost	0.9727	0.1245	0.8076	0.0148
	LightGBM	0.9718	0.0974	0.8071	0.0149
	OCT-H (Optimal)	0.9733	0.1750	0.7918	0.0211
	OCT-H (Heuristic)	0.9718	0.0958	0.7924	0.0151

Note: RF, XGBoost and LightGBM are trained on both binary and original datasets, and we only report the best results on the test set. The AUC and F1 scores of RF, XGBoost and LightGBM on all original datasets are better than those on binarized datasets.

interpretable but also highly accurate. Compared to previous classification tree models, OCT-H demonstrates superior accuracy due to its utilization of hyperplane splits for data partitioning and scalability. Additionally, this model offers great flexibility, enabling the inclusion of various constraints and objectives to address specific credit scoring challenges associated with imbalanced data. Two variants of OCT-H, namely CSOCT-H and OCT-H-F1, are proposed as solutions for credit scoring with imbalanced data.

Unlike CART, which is grown in a greedy manner, OCT-H is learned by optimizing an objective that considers both predictive performance and model complexity. To enable this model to handle large-scale datasets, we propose two methods to accelerate the solving process. Experimental results demonstrate that the proposed models perform

better than LR and DT in terms of interpretability and predictive performance. Furthermore, OCT-H significantly outperforms RF and exhibits comparable performance to “black-box” models such as LightGBM and XGBoost on most of the datasets, while retaining a high level of interpretability that these models lack. This conclusion highlights the promise and feasibility of designing fully transparent and interpretable machine learning models to address high-risk problems.

However, the OCT-H model also has limitations that must be acknowledged. Notably, on the Lending dataset, the AUCs of OCT-H models are similar to those of LR, while there are significant differences when compared to LightGBM and XGBoost. This finding highlights potential challenges in generalizing the model to predict future data, which warrants further investigation. Additionally, our data sample reduction method may result in the loss of important information from the original dataset, potentially leading to a suboptimal learned model. Furthermore, OCT-H is an NP-hard problem, meaning our proposed scalability methods may not be applicable to very large (over 1,000,000 samples) and highly sparse datasets.

Therefore, our future research will focus on developing more efficient algorithms to address these limitations, particularly in enhancing the scalability and predictive performance of OCT-H. Moreover, if interpretability is less critical than predictive performance, we plan to explore the development of an ensemble version of OCT-H, such as a boosted or bagged variant, to further enhance performance. This approach is particularly promising for datasets characterized by high class imbalance, where our models (OCT-H-F1 and CSOCT-H) have already shown excellence.

CRedit authorship contribution statement

Jiancheng Tu: Writing – original draft, Software. **Zhibin Wu:** Writing – review & editing.

Acknowledgments

The authors are grateful to the editors and anonymous reviewers for their insightful comments which have led to an improved version of this paper. Some ideas in the remarks are directly inspired by the reviewers' comments and observations. This work was supported by the National Natural Science Foundation of China under Grants 72371175 and 71971148 and by the Fundamental Research Funds for the Central Universities under Grants 2023ZY-SX020, 2023ZY-SX023, and SXPY202334. We are sincerely grateful to Professor Bart Baesens for generously providing us with the data sets (Bene1, Bene2, and UK) and for his invaluable professional advice.

References

- Aghaei, S., Gómez, A., & Vayanos, P. (2024). Strong optimal classification trees. *Operations Research*, <http://dx.doi.org/10.1287/opre.2021.0034>.
- Altman, E. I. (1968). Financial ratios, discriminant analysis and the prediction of corporate bankruptcy. *The Journal of Finance*, 23, 589–609.
- Andreeva, G., & Altman, E. I. (2021). The value of personal credit history in risk screening of entrepreneurs: Evidence from marketplace lending. *Journal of Financial Management, Markets and Institutions*, 9, Article 2150004.
- Aouad, A., Elmachtoub, A. N., Ferreira, K. J., & McNellis, R. (2023). Market segmentation trees. *Manufacturing & Service Operations Management*, 25, 648–667.
- Baesens, B., Höppner, S., & Verdonck, T. (2021). Data engineering for fraud detection. *Decision Support Systems*, 150, Article 113492.
- Baesens, B., & Smedts, K. (2023). Boosting credit risk models. *The British Accounting Review*, <http://dx.doi.org/10.1016/j.bar.2023.101241>.
- Baesens, B., Van Gestel, T., Viaene, S., Stepanova, M., Suykens, J., & Vanthienen, J. (2003). Benchmarking state-of-the-art classification algorithms for credit scoring. *Journal of the Operational Research Society*, 54, 627–635.
- Bahnsen, A. C., Aouada, D., & Ottersten, B. (2014). Example-dependent cost-sensitive logistic regression for credit scoring. In *2014 13th international conference on machine learning and applications* (pp. 263–269).
- Bellotti, T., & Crook, J. (2009). Support vector machines for credit scoring and discovery of significant features. *Expert Systems with Applications*, 36, 3302–3308.
- Bertsimas, D., & Dunn, J. (2017). Optimal classification trees. *Machine Learning*, 106, 1039–1082.
- Bertsimas, D., & Weismantel, R. (2005). Moment problems and semidefinite optimization. In *Optimization over integers*. Belmont, MA: Dynamic Ideas.
- Blanquero, R., Carrizosa, E., Molero-Río, C., & Morales, D. R. (2020). Sparsity in optimal randomized classification trees. *European Journal of Operational Research*, 284, 255–272.
- Bradford, J. P., Kunz, C., Kohavi, R., Brunk, C., & Brodley, C. E. (1998). Pruning decision trees with misclassification costs. In *European conference on machine learning* (pp. 131–136). Springer.
- Breiman, L. (2001). Random forests. *Machine Learning*, 45, 5–32.
- Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). *Classification and regression trees*. Monterey, CA: Wadsworth and Brooks.
- Bücker, M., Szepannek, G., Gosiewska, A., & Biecek, P. (2022). Transparency, auditability, and explainability of machine learning models in credit scoring. *Journal of the Operational Research Society*, 73, 70–90.
- Carrizosa, E., Molero-Río, C., & Romero Morales, D. (2021). Mathematical optimization in classification and regression trees. *Top*, 29, 5–33.
- Chaabane, I., Guermazi, R., & Hammami, M. (2020). Enhancing techniques for learning decision trees from imbalanced data. *Advances in Data Analysis and Classification*, 14, 677–745.
- Chang, Y.-C., Chang, K.-H., & Wu, G.-J. (2018). Application of eXtreme gradient boosting trees in the construction of credit risk assessment models for financial institutions. *Applied Soft Computing*, 73, 914–920.
- Chen, Y., Calabrese, R., & Martin-Barragan, B. (2024). Interpretable machine learning for imbalanced credit scoring datasets. *European Journal of Operational Research*, 312, 357–372.
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 785–794). New York, NY, USA: Association for Computing Machinery.
- Chen, C., Lin, K., Rudin, C., Shaposhnik, Y., Wang, S., & Wang, T. (2022). A holistic approach to interpretability in financial lending: Models, visualizations, and summary-explanations. *Decision Support Systems*, 152, Article 113647.
- Ciocan, D. F., & Mišić, V. V. (2022). Interpretable optimal stopping. *Management Science*, 68, 1616–1638.
- DeLong, E. R., DeLong, D. M., & Clarke-Pearson, D. L. (1988). Comparing the areas under two or more correlated receiver operating characteristic curves: A nonparametric approach. *Biometrics*, 837–845.
- Demajo, L. M., Vella, V., & Dingli, A. (2020). Explainable ai for interpretable credit scoring. *arXiv preprint arXiv:2012.03749*.
- Dietterich, T. G. (1998). Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10, 1895–1923.
- Doshi-Velez, F., & Kim, B. (2017). Towards a rigorous science of interpretable machine learning. *arXiv preprint arXiv:1702.08608*.
- Dumitrescu, E., Hué, S., Hurlin, C., & Tokpavi, S. (2022). Machine learning for credit scoring: Improving logistic regression with non-linear decision-tree effects R. *European Journal of Operational Research*, 297, 1178–1192.
- Fayyad, U. M., & Irani, K. B. (1993). Multi-interval discretization of continuous-valued attributes for classification learning. In *International joint conference on artificial intelligence*, Vol. 93 (pp. 1022–1029). Citeseer.
- Financial Stability Board (2017). Artificial intelligence and machine learning in financial services – market developments and financial stability implications. URL: <https://www.fsb.org/wp-content/uploads/P011117.pdf>.
- Guidotti, R., Monreale, A., Ruggieri, S., Pedreschi, D., Turini, F., & Giannotti, F. (2018). Local rule-based explanations of black box decision systems. *arXiv preprint arXiv:1805.10820*.
- Gunnarsson, B. R., vanden Broucke, S., Baesens, B., Óskarsdóttir, M., & Lemahieu, W. (2021). Deep learning for credit scoring: Do or don't? *European Journal of Operational Research*, 295, 292–305.
- Gurobi Optimization, L. (2024). Gurobi optimizer reference manual. URL: <https://www.gurobi.com>. Version 11.0.
- He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21, 1263–1284.
- Höppner, S., Baesens, B., Verbeke, W., & Verdonck, T. (2022). Instance-dependent cost-sensitive learning for detecting transfer fraud. *European Journal of Operational Research*, 297, 291–300.
- IBM (2023). *IBM ILOG CPLEX Optimization Studio*. Armonk, NY: IBM, URL: <https://www.ibm.com/products/ilog-cplex-optimization-studio>. Version 22.1.
- Iranmehr, A., Masnadi-Shirazi, H., & Vasconcelos, N. (2019). Cost-sensitive support vector machines. *Neurocomputing*, 343, 50–64.
- Jiang, C., Lu, W., Wang, Z., & Ding, Y. (2023). Benchmarking state-of-the-art imbalanced data learning approaches for credit scoring. *Expert Systems with Applications*, 213, Article 118878.
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., et al. (2017). Lightgbm: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30.
- Kellner, R., Nagl, M., & Rösch, D. (2022). Opening the black box—quantile neural networks for loss given default prediction. *Journal of Banking & Finance*, 134, Article 106334.

- Kim, J., Choi, K., Kim, G., & Suh, Y. (2012). Classification cost: An empirical comparison among traditional classifier, Cost-Sensitive Classifier, and MetaCost. *Expert Systems with Applications*, 39, 4013–4019.
- Krawczyk, B., Woźniak, M., & Schaefer, G. (2014). Cost-sensitive decision tree ensembles for effective imbalanced classification. *Applied Soft Computing*, 14, 554–562.
- Laurent, H., & Rivest, R. L. (1976). Constructing optimal binary decision trees is NP-complete. *Information Processing Letters*, 5, 15–17.
- Lessmann, S., Baesens, B., Seow, H. V., & Thomas, L. C. (2015). Benchmarking state-of-the-art classification algorithms for credit scoring: An update of research. *European Journal of Operational Research*, 247, 124–136.
- Liu, Y., Baals, L. J., Osterrieder, J., & Hadji-Misheva, B. (2024). Leveraging network topology for credit risk assessment in P2P lending: A comparative study under the lens of machine learning. *Expert Systems with Applications*, 252, Article 124100.
- Liu, W., Fan, H., & Xia, M. (2023). Tree-based heterogeneous cascade ensemble model for credit scoring. *International Journal of Forecasting*, 39, 1593–1614.
- Liu, W., Fan, H., Xia, M., & Xia, M. (2022). A focal-aware cost-sensitive boosted tree for imbalanced credit scoring. *Expert Systems with Applications*, 208, Article 118158.
- Lohmann, C., & Ohliger, T. (2019). The total cost of misclassification in credit scoring: A comparison of generalized linear models and generalized additive models. *Journal of Forecasting*, 38, 375–389.
- Lomax, S., & Vadera, S. (2013). A survey of cost-sensitive decision tree induction algorithms. *ACM Computing Surveys*, 45, 1–35.
- Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30.
- Luo, J., Yan, X., & Tian, Y. (2020). Unsupervised quadratic surface support vector machine with application to credit risk assessment. *European Journal of Operational Research*, 280, 1008–1017.
- Medina-Olivares, V., Calabrese, R., Crook, J., & Lindgren, F. (2023). Joint models for longitudinal and discrete survival data in credit scoring. *European Journal of Operational Research*, 307, 1457–1473.
- Miguéis, V. L., Benoit, D. F., & Van den Poel, D. (2013). Enhanced decision support in credit scoring using Bayesian binary quantile regression. *Journal of the Operational Research Society*, 64, 1374–1383.
- Miller, G. A. (1956). The magical number seven, plus or minus two: Some limits on our capacity for processing information. *Psychological Review*, 63, 81.
- Molnar, C. (2020). *Interpretable machine learning*. Lulu. com.
- Moscato, V., Picariello, A., & Sperli, G. (2021). A benchmark of machine learning approaches for credit score prediction. *Expert Systems with Applications*, 165, Article 113986.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Petrides, G., Moldovan, D., Coenen, L., Guns, T., & Verbeke, W. (2022). Cost-sensitive learning for profit-driven credit scoring. *Journal of the Operational Research Society*, 73, 338–350.
- Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). “Why should I trust you?: Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 1135–1144). New York, NY, USA: Association for Computing Machinery.
- Ribeiro, M. T., Singh, S., & Guestrin, C. (2018). Anchors: High-precision model-agnostic explanations. In *Proceedings of the AAAI conference on artificial intelligence: Vol. 32*.
- Ross, A. S., Hughes, M. C., & Doshi-Velez, F. (2017). Right for the right reasons: Training differentiable models by constraining their explanations. In *Proceedings of the 26th international joint conference on artificial intelligence* (pp. 2662–2670).
- Rozo, B. J. G., Crook, J., & Andreeva, G. (2023). The role of web browsing in credit risk prediction. *Decision Support Systems*, 164, Article 113879.
- Rudin, C. (2019). Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1, 206–215.
- Siddiqi, N. (2012). *Credit risk scorecards: Developing and implementing intelligent credit scoring. volume 3*. John Wiley & Sons.
- Slack, D., Hilgard, S., Jia, E., Singh, S., & Lakkaraju, H. (2020). Fooling lime and shap: Adversarial attacks on post hoc explanation methods. In *Proceedings of the AAAI/ACM conference on AI, ethics, and society* (pp. 180–186).
- Teles, G., Rodrigues, J. J., Saleem, K., Kozlov, S., & Rabêlo, R. A. (2020). Machine learning and decision support system on credit scoring. *Neural Computing and Applications*, 32, 9809–9826.
- Thomas, L., Crook, J., & Edelman, D. (2002). *Credit scoring and its applications*. SIAM.
- Verbraken, T., Bravo, C., Weber, R., & Baesens, B. (2014). Development and application of consumer credit scoring models using profit-based classification measures. *European Journal of Operational Research*, 238, 505–513.
- Wang, Z., Crook, J., & Andreeva, G. (2020). Reducing estimation risk using a Bayesian posterior distribution approach: Application to stress testing mortgage loan default. *European Journal of Operational Research*, 287, 725–738.
- Wang, T., & Lin, Q. (2021). Hybrid predictive models: When an interpretable model collaborates with a black-box model. *Journal of Machine Learning Research*, 22, 1–38.
- West, D. (2000). Neural network credit scoring models. *Computers & Operations Research*, 27, 1131–1152.
- Yotsawat, W., Wattuya, P., & Srivihok, A. (2021). A novel method for credit scoring based on cost-sensitive neural network ensemble. *IEEE Access*, 9, 78521–78537.